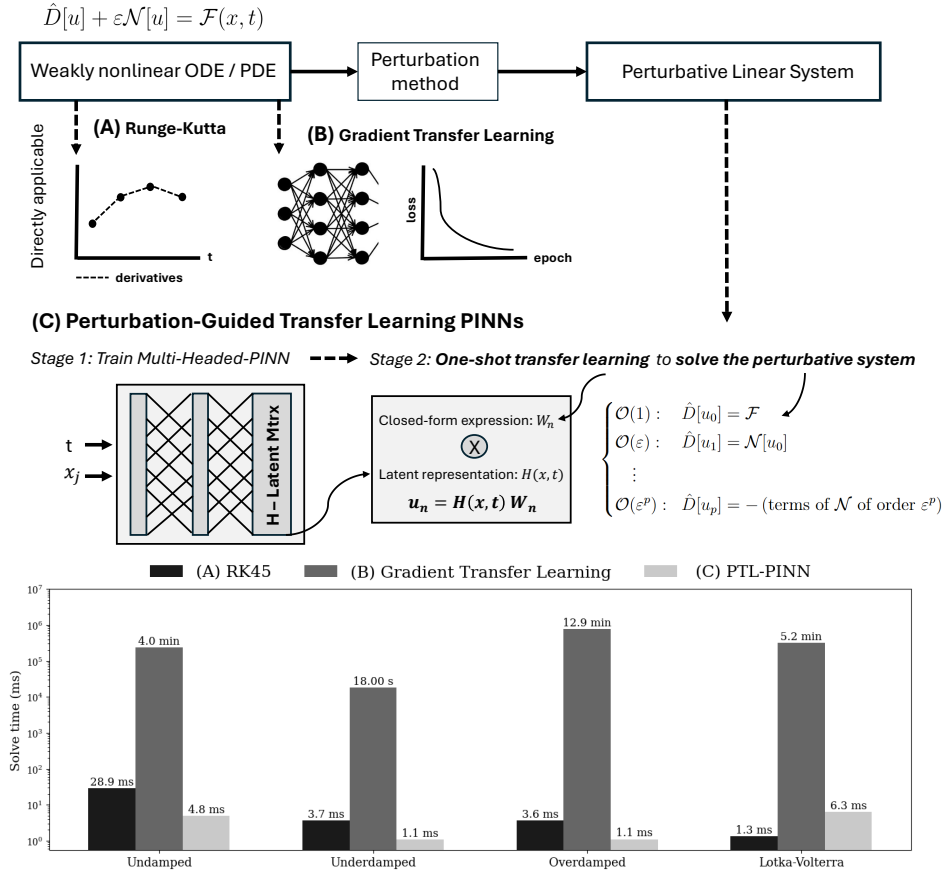


Graphical Abstract

PTL-PINNs: Perturbation-Guided Transfer Learning with Physics-Informed Neural Networks for Nonlinear Systems

Duarte Alexandrino, Ben Moseley, Pavlos Protopapas



Highlights

PTL-PINNs: Perturbation-Guided Transfer Learning with Physics-Informed Neural Networks for Nonlinear Systems

Duarte Alexandrino, Ben Moseley, Pavlos Protopapas

- We introduce a Perturbation-Guided Transfer Learning framework for PINNs (PTL-PINNs), which combines perturbation theory with one-shot transfer learning to efficiently solve the sequence of linear systems that approximate weakly nonlinear ODEs and PDEs.
- We benchmark PTL-PINNs across a broad class of weakly nonlinear systems, including second-order and coupled ODEs, as well as PDEs that are first-order in time and first-order or second-order in space.
- We show that PTL-PINNs can solve weakly nonlinear systems up to an order of magnitude faster than Runge–Kutta methods by generalizing to new equations via closed-form updates of the output weights, avoiding gradient-based optimization.
- We derive general n th-order perturbation equations and forcing functions using both standard and Lindstedt–Poincaré methods. These derivations enable PTL-PINNs to solve perturbative systems to higher orders and guide the selection of pretraining equations.
- We demonstrate how perturbation methods can be systematically integrated with PINNs. We show that PTL-PINNs reproduce known perturbation-theory failure modes and that the number of perturbation basis functions scales linearly with runtime and logarithmically with error.

PTL-PINNs: Perturbation-Guided Transfer Learning with Physics-Informed Neural Networks for Nonlinear Systems

Duarte Alexandrino^{a,1}, Ben Moseley^a, Pavlos Protopapas^b

^aDepartment of Earth Science and Engineering, Imperial College, Exhibition Road, London, SW7 2AZ, England

^bHarvard John A. Paulson School Of Engineering And Applied Sciences, 150 Western Ave, Boston, MA 02134, United States

Abstract

Accurately and efficiently solving nonlinear differential equations is crucial for modeling dynamic behavior across science and engineering. Physics-Informed Neural Networks (PINNs) have emerged as a powerful solution that embeds physical laws in training by enforcing equation residuals. However, these struggle to model nonlinear dynamics, suffering from limited generalization across problems and long training times. To address these limitations, we propose a perturbation-guided transfer learning framework for PINNs (PTL-PINN), which integrates perturbation theory with transfer learning to efficiently solve nonlinear equations. Unlike gradient-based transfer learning, PTL-PINNs solve an approximate linear perturbative system using closed-form expressions, enabling rapid generalization with the time complexity of matrix–vector multiplication. We show that PTL-PINNs achieve accuracy comparable to various Runge–Kutta methods, with computational speeds up to one order of magnitude faster. To benchmark performance, we solve a broad set of problems, including nonlinear oscillators across various damping regimes, the equilibrium-centered Lotka–Volterra system, the KPP–Fisher and the Wave equation. Since perturbation theory sets the accuracy bound of PTL-PINNs, we systematically evaluate its practical applicability. This work connects long-standing perturbation methods with PINNs, demonstrating how perturbation theory can guide foundational models to solve nonlinear

¹Corresponding author email: d.alexandrino2010@gmail.com

systems with speeds comparable to those of classical solvers.

Keywords: Physics-Informed Neural Networks, PINNs, Nonlinear differential equations, Perturbation Theory, Transfer learning, Computational efficiency.

PACS: 07.05.Mh, 07.05.Tp

1. Introduction

Differential equations are ubiquitous in modeling real-world phenomena. A wide range of scientific and engineering problems can be reduced to formulating and solving differential equations. However, most nonlinear differential equations do not admit analytical solutions. The standard approach to solving them is numerical methods such as the finite difference, finite element or finite volume method [1]. These provide robust approximations but require iterative time-stepping schemes, needing to be rerun for every new equation. This makes large-scale simulations computationally expensive and limits their practicality in settings that demand repeated evaluations of nonlinear differential equations, such as design optimization or Bayesian inference [2, 3].

There has been a growing trend towards using neural networks to accelerate simulations in recent years. These have been used to accelerate and improve the results of numerical solvers [4, 5, 6]. Moreover, they have been used to entirely replace them through architectures such as Neural Operators [7] and Physics-Informed Neural Networks (PINNs) [8, 9]. While Neural Operators learn the mapping between input and solution function spaces purely from data [10, 11, 12], PINNs leverage automatic differentiation to enforce the residuals of the governing differential equations directly in the loss function, making them particularly effective in low-data regimes. In this process, they learn the functional representations of equations, offering rapid forward pass evaluations [13].

PINNs have been successfully applied to solve nonlinear differential equations across various areas, including fluid dynamics, wave propagation and chaotic systems [14, 15, 16]. Despite being good solvers for relatively trivial problems, they easily fail to learn relevant physical phenomena for even slightly more complex equations. For nonlinear oscillatory systems, such as those considered in this work, PINNs tend to learn higher frequencies more slowly than lower frequencies, which is known as the spectral bias [17].

The failure modes of PINNs are often not due to the lack of expressivity in the neural network architecture, but rather to a loss landscape being highly non-convex and hard to optimize [18]. More fundamentally, the practical applicability of PINNs is limited by poor generalizability across equations. In the standard approach, a PINN is trained to minimize the residual of a specific differential equation and its associated initial/boundary conditions. Consequently, the learned weights fail to generalize when physical parameters and initial/boundary conditions change [19].

Transfer learning has been used to address this limitation [20, 21, 22]. This method involves reusing a pretrained neural network to infer the solution for a different equation. In practice, it is performed by initializing a new network with the weights of the pretrained PINN and optimizing the weights of the last layer(s) based on the loss function of a new and closely related problem. Since the network’s weights already have encoded the general features of the underlying physics, this requires significantly fewer iterations than training a new model. This process still relies on gradient-based optimization to update model weights, which remains orders of magnitude slower than classical numerical solvers.

A faster alternative is one-shot transfer learning [23]. This approach leverages a Multi-Headed-PINN architecture, which approximates equations of similar form with a shared latent representation [24]. In one-shot transfer learning, the solution to a new equation is obtained from a closed-form expression rather than through gradient descent, reducing inference to matrix operations. However, one-shot transfer learning is only applicable to linear differential equations, since their loss functions are convex with respect to the weights. Recent work with PINNs has begun to explore perturbative strategies to relax this linearity restriction by approximating nonlinear problems through structured linear expansions [25, 26]. These approaches motivate the idea that weakly nonlinear systems may be tractable via linear surrogate problems, but they do not yet provide a general framework that combines perturbation theory with reusable latent representations and closed-form inference.

In this work, we introduce *Perturbation-Guided Transfer Learning for Physics-Informed Neural Networks* (PTL-PINNs), a general framework that extends one-shot transfer learning to weakly nonlinear differential equations. The central idea is to reformulate a nonlinear problem as a hierarchy of linear differential equations derived from perturbation theory [27] and to solve each linear subproblem via one-shot transfer learning using a shared latent rep-

representation learned by a Multi-Headed-PINN. While perturbative one-shot approaches have been explored for specific systems, PTL-PINNs provide a unified and reusable framework that applies across ordinary, coupled and partial differential equations. The framework is designed to systematically incorporate different perturbative formulations, which allow known limitations of perturbation theory, particularly in oscillatory systems, to be addressed within a common computational setting. By replacing gradient-based optimization with closed-form updates of the output weights at each perturbation order, PTL-PINNs enable rapid generalization to new equations, achieving runtimes competitive with classical numerical solvers from the Runge–Kutta family [28].

Various formulations of perturbation methods have been widely applied in fields ranging from perturbed eigenvalue problems in quantum mechanics to celestial mechanics [29, 30]. For oscillatory equations, the standard perturbation method fails to account for the frequency shift introduced by the nonlinearity. To address this limitation, the Lindstedt–Poincaré method incorporates frequency corrections directly into the perturbation expansion, systematically eliminating non-physical terms. This approach has since become a foundational tool in celestial mechanics, notably applied to the analysis of the circular restricted three-body problem [31, 32]. Beyond astronomy, it has proven valuable in engineering, for example, in vibration analyses of laminated plates [33] and in biology, where it has been applied to predator–prey models [34].

Within PTL-PINNs, such alternative perturbative formulations play a key role in enabling stable and accurate one-shot transfer learning for oscillatory systems and are incorporated naturally into the same computational framework. Using this framework, we demonstrate accurate and efficient solutions across ordinary, coupled and partial weakly nonlinear differential equations. The resulting solutions are expressed as linear combinations of a shared latent representation extracted from a pretrained Multi-Headed-PINN, which directly informs the choice of pretraining equations through the structure of the underlying perturbative system.

The primary contributions of this work are summarized as follows:

- We introduce a Perturbation-Guided Transfer Learning framework for PINNs (PTL-PINNs), which combines perturbation theory with one-shot transfer learning to efficiently solve the sequence of linear systems that approximate weakly nonlinear ODEs and PDEs.

- We benchmark PTL-PINNs across a broad class of weakly nonlinear systems, including second-order and coupled ODEs, as well as PDEs that are first-order in time and first-order or second-order in space.
- We show that PTL-PINNs can solve weakly nonlinear systems up to an order of magnitude faster than Runge–Kutta methods by generalizing to new equations via closed-form updates of the output weights, avoiding gradient-based optimization.
- We derive general n th-order perturbation equations and forcing functions using both standard and Lindstedt–Poincaré methods. These derivations enable PTL-PINNs to solve perturbative systems to higher orders and guide the selection of pretraining equations.
- We demonstrate how perturbation methods can be systematically integrated with PINNs. We show that PTL-PINNs reproduce known perturbation-theory failure modes and that the number of perturbation basis functions scales linearly with runtime and logarithmically with error.

Viewed broadly, this paper unifies perturbation theory, including the standard perturbation and Lindstedt–Poincaré methods, with PINNs, demonstrating how classical analytical structure can be leveraged by modern neural solvers to scale across nonlinear ODEs and PDEs.

The rest of the work is organized as follows. First, we provide an overview of related work in Section 2. Then, we detail the PTL-PINNs methodology in Section 3, describing perturbation theory and one-shot transfer learning. Next, we show its accuracy across a variety of differential equations. Then, we benchmark the computational efficiency of PTL-PINNs in Section 5. Finally, we discuss the limitations of PTL-PINNs and present future directions in Sections 6 and 7.

2. Related Work

One-shot transfer learning has been proposed to accelerate PINNs for linear equations [23], but it is not directly applicable to nonlinear problems. Initial proof-of-concept studies have shown that perturbation theory can be used to extend one-shot transfer learning for weakly nonlinear ODEs [25] and PDEs [26] by approximating them with systems of linear equations.

The idea of linearizing a problem to accelerate PINNs has also been explored through extreme learning machines, which randomly initialize the hidden-layer weights and only learn the output layer, enabling training via a fast linear optimization process [35, 36].

Other branches of perturbation theory have been leveraged to develop enhanced PINN frameworks. A prominent example is the reformulation as singular perturbation problems of thin boundary layer problems, which are known to challenge both PINNs and traditional numerical methods. Boundary Layer PINNs (BL-PINNs) explicitly incorporate this structure to better resolve boundary layer behavior [37], with subsequent works demonstrating improved performance and alternative formulations [38, 39]. The perturbative Method of Multiple Scales has also been employed to identify appropriate scaling parameters for PINN inputs, leading to more robust multiscale learning [40].

Despite promising approaches that combine perturbation theory with PINNs, the existing literature on perturbation methods primarily focuses on low-order analytical solutions [29, 32, 33] or relies on semi-symbolic approaches [34]. Consequently, there is a lack of generalizable frameworks that can systematically integrate PINNs into perturbative analyses. This gap motivates the development of the PTL-PINN framework, which is designed to be broadly applicable to one-shot transfer learning with PINNs.

3. Methodology

3.1. Notation and conventions

We summarize the notation and indexing conventions used throughout the paper to improve clarity and consistency across perturbative and learning components.

We use $u(t)$ for ODEs and $u(x, t)$ for PDEs. When needed, bold symbols denote vector-valued states. We use $\hat{D}$ and $\mathcal{N}$ to denote abstract linear and nonlinear operators in the general perturbation formulation. For ODEs, the resulting linear subproblems are written in first-order matrix form using operators A and B , with forcing denoted by $F(t)$. For PDEs, the forcing term is written as $f(x, t)$. When the system is vector-valued, bold symbols (e.g. $\mathbf{u}$, $\mathbf{F}$) are used.

Indices and dimensions follow standard conventions. The perturbation order is indexed by $n = 0, \dots, p$. Training equations (heads) are indexed by $i = 1, \dots, k$. Collocation points are indexed by i (interior), j (initial) and

h (boundary). The latent representation H has dimension $\mathbb{R}^{n \times m}$, where N is the number of collocation points and m the latent width. Head weights $W_\theta \in \mathbb{R}^{m \times r}$ map the latent space to r state components (or derivatives). All summations over $t \in T$ denote discrete sums over collocation points.

Throughout this work, t denotes physical time. In the Lindstedt–Poincaré method, we introduce the rescaled time $\tau = \omega t$ to absorb nonlinear frequency shifts. Here, ω_0 denotes the linear natural frequency, ω the nonlinear corrected frequency and ω_n its n th-order perturbative correction. For externally forced systems, Ω_k denotes prescribed forcing frequencies.

3.2. Perturbation theory

3.2.1. Introduction

Perturbation theory deals with weakly nonlinear systems, generally described as:

$$\hat{D}[u] + \varepsilon \mathcal{N}[u] = \mathcal{F}(\mathbf{x}, t), \quad (1)$$

Here, $\hat{D}$ denotes a linear differential operator in time and/or space, $\mathcal{N}$ a polynomial nonlinear operator and $\mathcal{F}$ represents a known forcing term incorporating external inputs.

The small parameter ε quantifies the deviation from the linear regime. In perturbation theory, the solution is expanded in a series around this coefficient and can be truncated at p terms:

$$u(x, t) \approx u_0(x, t) + \varepsilon u_1(x, t) + \cdots + \varepsilon^p u^p(x, t) \quad (2)$$

Substituting the expansion in equation 2 into equation 1 and collecting terms of equal powers of ε yields a sequence of linear differential equations represented in the system of equations 3. The solutions to these linear equations yield the successive correction terms defined in equation 2:

$$\begin{cases} \mathcal{O}(1) : & \hat{D}[u_0] = \mathcal{F} \\ \mathcal{O}(\varepsilon) : & \hat{D}[u_1] = \mathcal{N}[u_0] \\ & \vdots \\ \mathcal{O}(\varepsilon^p) : & \hat{D}[u_p] = -(\text{terms of } \mathcal{N} \text{ of order } \varepsilon^p) \end{cases} \quad (3)$$

This perturbative structure transforms the original nonlinear problem into a sequence of linear subproblems that share the same linear operator but differ in their forcing terms. This structural property is the key feature exploited by the proposed PTL-PINN framework.

3.2.2. Canonical Oscillator

To motivate the need for alternative perturbation methods, such as the Lindstedt–Poincaré method, we briefly consider the canonical nonlinear oscillator with monomial nonlinearity with power q :

$$u'' + 2\omega_0\zeta u' + \omega_0^2 u + \varepsilon u^q = \sum_{k=0}^m \Gamma_k \cos(\Omega_k t) \quad (4)$$

The linear system that results from inserting the perturbation expansion of equation 2 into equation 4 is the following, where the forcing is given by the multinomial coefficient sum:

$$\left\{ \begin{array}{l} \mathcal{O}(1) : \quad \ddot{u}_0 + 2\zeta\omega_0\dot{u}_0 + \omega_0^2 u_0 = \sum_{k=0}^m \Gamma_k \cos(\Omega_k t) \\ \mathcal{O}(\varepsilon) : \quad \ddot{u}_1 + 2\zeta\omega_0\dot{u}_1 + \omega_0^2 u_1 = -u_0^q \\ \vdots \\ \mathcal{O}(\varepsilon^n) : \quad \ddot{u}_n + 2\zeta\omega_0\dot{u}_n + \omega_0^2 u_n = \sum_{\substack{k_0+k_1+\dots+k_p=q \\ \sum_{i=0}^p i k_i = n-1}} \frac{q!}{k_0! k_1! \dots k_p!} \prod_{i=0}^p u_i^{k_i} \\ \vdots \end{array} \right. \quad (5)$$

3.2.3. Limitations of standard perturbation for oscillatory systems

The solution of the system of equations 5 for the undamped case ($\zeta = 0$) has terms of the form $t^n \sin t$. These terms are non-physical, as they imply unbounded growth in time, despite the underlying system being energy-conserving. This phenomenon is called resonance and occurs in undamped oscillatory systems when the forcing term has a non-zero projection onto the null space of the linear operator, that is, when it shares the same functional form as the homogeneous solution.

This breakdown reflects a violation of the solvability conditions of the linear subproblems and limits the validity of standard perturbation expansions for oscillatory dynamics. This highlights the need for perturbation formulations that explicitly enforce solvability conditions. While damping suppresses true resonance, near-resonant behavior can still arise in weakly damped regimes due to small-denominator effects [41].

3.2.4. Lindstedt–Poincaré perturbation method

To ensure the solvability conditions are met when solving undamped oscillators, such as the canonical oscillator and the equilibrium-centered Lotka–Volterra system, we adopt the Lindstedt–Poincaré method within the PTL-PINN framework. This prevents resonance, removing non-physical terms from the perturbation series, by introducing a rescaled time variable $\tau = \omega t$, where the frequency ω is itself expanded as a perturbative series:

$$\omega \approx \omega_0 + \varepsilon\omega_1 + \cdots + \varepsilon^p\omega_p, \quad \omega \in \mathbb{R} \quad (6)$$

This additional expansion adds flexibility to the series by introducing frequency corrections at each order, which are chosen to eliminate resonant forcing terms. After the time rescaling of the nonlinear equation, each perturbation order yields a modified forcing term that depends on lower-order solutions and the unknown frequency correction.

In our approach, the Lindstedt–Poincaré system is solved numerically rather than symbolically. At each perturbation order $n > 0$, the frequency correction ω_n is computed by enforcing a solvability condition that removes the projection of the forcing onto the homogeneous solution of the linear operator. Once ω_n is determined, the corresponding linear equation for u_n is solved using PTL-PINNs via one-shot transfer learning, exactly as in the standard perturbation setting.

After calculating the frequency and solution corrections, we sum our series to obtain the final solutions $u \approx u_0 + \cdots + \varepsilon^n u_n$ and $\omega = \omega_0 + \cdots + \varepsilon^n \omega_n$. Since we performed the calculations with our original time array, our solution is composed of terms with frequency linear in t and not in τ . By plotting our solution for t/ω , effectively, the frequency of each term is multiplied by ω , ensuring our result captures the nonlinear frequency.

Importantly, despite the introduction of frequency corrections, the Lindstedt–Poincaré formulation preserves the hierarchical structure required by PTL-PINNs: each perturbation order remains a linear problem sharing the same linear operator, differing only in its forcing term. This allows oscillatory systems to be treated within the same perturbation-guided transfer learning pipeline as non-oscillatory systems, without introducing divergent terms.

Detailed derivations of the Lindstedt–Poincaré perturbative systems for the undamped oscillator and the equilibrium-centered Lotka–Volterra equations are provided in the Appendix.

3.3. Perturbation-guided transfer learning (PTL-PINNs)

Both standard perturbation expansions and alternative formulations such as the Lindstedt–Poincaré method yield, at each order, linear differential equations sharing a common linear operator but differing in their forcing terms.

3.3.1. Multi-Headed-PINN

One-shot transfer learning was proposed by Desai et al. [23] for linear differential equations of the general form $\hat{D}u(x, t) = f(x, t)$, where $\hat{D}$ is a linear differential operator. This method requires a Multi-Headed-PINN to be pretrained so that it approximates equations of similar form with a shared latent representation, $H(x, t)$. A Multi-Headed-PINN consists of a shared feedforward backbone with k output training equations (heads), each corresponding to a parameter set θ^{train} .

Unlike traditional PINN methods, which minimize the loss function of a single equation, the Multi-Headed-PINN pretraining minimizes the average loss across all training equations (heads), defined as $\mathcal{L}_{\text{Multi-Head}} = \frac{1}{k} \sum_{i=1}^k \mathcal{L}_i$.

Defining interior collocation points as $\{(x_i, t_i)\}_{i=1}^{N_{\text{int}}}$, initial condition points as $\{x_j\}_{j=1}^{N_{\text{ic}}}$, and boundary points as $\{t_h\}_{h=1}^{N_{\text{bc}}}$, initial condition as $IC(x)$ and boundary condition as $BC(t)$, the loss for a single equation ($\mathcal{L}_k$) parameterized by θ is given by:

$$\begin{aligned} \mathcal{L}_k = & w_{\text{pde}} \sum_{i=1}^{N_{\text{int}}} (\hat{D}u_{\theta}(x_i, t_i) - f_{\theta}(x_i, t_i))^2 \\ & + w_{\text{ic}} \sum_{j=1}^{N_{\text{ic}}} (u_{\theta}(x_j, 0) - IC_{\theta}(x_j))^2 \\ & + w_{\text{bc}} \sum_{h=1}^{N_{\text{bc}}} (u_{\theta}(x_h, t_h) - BC_{\theta}(t_h))^2 \end{aligned} \quad (7)$$

In the case of ODEs of order r , we first rewrite them as a simpler first-order system of the form:

$$A \dot{\mathbf{u}}_{\theta}(t) + B \mathbf{u}_{\theta}(t) = F_{\theta}(t), \quad (8)$$

Here, $\mathbf{u}_{\theta}(t)$ is the vector of derivatives of u_{θ} from order 0 to $r - 1$ and $\dot{\mathbf{u}}_{\theta}(t)$ is the vector of derivatives from order 1 to r . The matrices A and B

encode the linear coupling between these components and $F_\theta(t)$ is the forcing term.

We adopt this formulation because it provides a unified representation for higher-order and coupled ODE systems within the same architecture. As a result, the canonical oscillator and the coupled Lotka–Volterra system have transformed into the same structural form, allowing both to be handled by the same PTL-PINN model.

In practice, for r -dimensional systems, we split the final latent vector of the shared backbone into r sub-vectors and apply the same linear head to each half. An alternative but mathematically equivalent formulation, where the latent vector is kept intact and two separate linear heads are used, is described in [23].

The general architecture we use for pretraining a Multi-Headed-PINN in this manuscript, along with the inference process, is illustrated in Fig. 1. The input $\mathbf{x} \in \mathbb{R}^{n \times d+1}$ is transformed through a feature engineering stage into $\mathbf{I} \in \mathbb{R}^{n \times f}$, which is passed through multiple hidden layers to construct the latent matrix $\mathbf{H} \in \mathbb{R}^{n \times rm}$. Finally, through multiplication with $\mathbf{H}$, the head weights $\mathbf{W}_\theta \in \mathbb{R}^{m \times r}$ map the latent space to r state components (or derivatives).

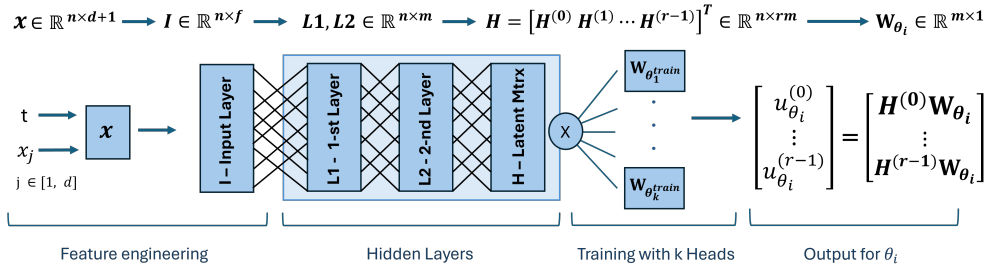


Figure 1: Multi-Headed-PINN architecture used for one-shot transfer learning. A shared neural backbone learns a latent representation $\mathbf{H}(x, t)$ across k training equations, while linear output heads represented by matrices $\mathbf{W}$ (head weights) map this representation to solution components for each equation. At inference time, new linear problems are solved by computing closed-form output weights without gradient-based optimization.

3.3.2. One-shot transfer learning

In this subsection, we outline the derivation of the closed-form expressions used in one-shot transfer learning, following Desai et al. [23]. After pretraining a Multi-Headed-PINN, the latent representation $H(t, x)$ is extracted and

the training-specific head weights $W_{\theta_i^{\text{train}}}$ are discarded. As shown in Fig. 1, the solution for each order parameterized by θ can be obtained by multiplying our parameter-independent $H(t, x)$ by a head layer W_θ . For a new linear differential equation, we parameterize the solution as $\mathbf{u}(x, t) = H(x, t)W_\theta$ and substitute this form into the loss defined in equation 7.

$$\begin{aligned} \mathcal{L} = & w_{\text{pde}} \sum_{i=1}^{N_{\text{int}}} (\hat{D}H(x_i, t_i)W_\theta - f_\theta(x_i, t_i))^2 \\ & + w_{\text{ic}} \sum_{j=1}^{N_{\text{ic}}} (H(x_j, 0)W_\theta - IC_\theta(x_j))^2 \\ & + w_{\text{bc}} \sum_{h=1}^{N_{\text{bc}}} (H(x_h, t_h)W_\theta - BC_\theta(t_h))^2 \end{aligned} \quad (9)$$

Since the loss is quadratic in W_θ , the optimal head weights are obtained by solving the normal equations corresponding to the condition $\partial\mathcal{L}/\partial W_\theta = 0$. Following the derivation of one-shot transfer learning in [23], it is possible to compute W_θ for this new equation with equation 10.

$$\begin{aligned} W_\theta = M^{-1} \left(\sum_{i=1}^{N_{\text{int}}} (\hat{D}H(x_i, t_i))^T f(x_i, t_i) \right. \\ \left. + \sum_{j=1}^{N_{\text{ic}}} H^T(x_j, 0) IC(x_j) \right. \\ \left. + \sum_{h=1}^{N_{\text{bc}}} H^T(x_h, t_h) BC(t_h) \right) \end{aligned} \quad (10)$$

Here, $M \in \mathbb{R}^{m \times m}$ is the normal matrix arising from the quadratic loss in equation 7 and depends only on the latent representation H , the collocation set and on the differential operator $\hat{D}$. Its explicit form is provided in [23]. Since M is independent of the forcing F and the perturbation order, M^{-1} can be computed once and reused across all perturbation solves. From equation 10, the solution can be obtained with time complexity of a matrix inversion or as fast as matrix-vector multiplications if M^{-1} is reused, which can be done within the same perturbation system or between equations with same $\hat{D}$ but different initial/boundary conditions, nonlinearity or forcing.

Obtaining equation 10 is only possible for linear differential equations. Only these systems lead to a convex optimization problem in the head weights of W_θ , allowing an analytical solution of the $\frac{d\mathcal{L}}{dW_\theta} = 0$. Consequently, one-shot transfer learning is limited to linear differential equations and PTL-PINNs are needed to generalize it to weakly nonlinear differential equations.

3.3.3. Two stages of PTL-PINNs

The PTL-PINN framework consists of two main stages: (1) pretraining a Multi-Headed-PINN on a set of linear differential equations and (2) using the learned latent representation to iteratively solve each linear equation in the perturbative system via one-shot transfer learning.

Stage 1: Perturbation-guided pretraining of a Multi-Headed-PINN. The PTL-PINN solution is a weighted combination of the latent representation $H(x, t)$. This motivates the incorporation of the perturbation structure directly into the training process to ensure a meaningful latent representation is found. First, we analyze the structure of the perturbative system associated with the target nonlinear equation. This analysis guides the selection of a representative set of linear differential equations for pretraining the Multi-Headed-PINN. The analytical solutions of the zeroth-order equation guide the choice of the first subset of pretraining equations, while the structure of the first-order forcing informs the selection of additional equations. Finally, we include linear equations with more complex forcing terms that resemble higher-order perturbative contributions, enabling the latent space to generalize to later perturbation orders. The network architecture is selected to balance expressivity and stability across the chosen family of linear problems. The exact equations chosen in training alongside their justification for each system solved can be found in the Appendix.

Stage 2: Iterative perturbation-guided one-shot transfer learning. The solution at perturbation order n is obtained by solving a linear differential equation parameterized by θ_n . Each solution is computed by multiplying the parameter-independent latent representation $H(x, t)$ by an output head W_{θ_n} obtained via one-shot transfer learning. The final solution is constructed by summing contributions from all perturbation orders, which corresponds to a weighted combination of outputs from the shared latent space $H(t, x)$:

$$H(t) (W_{\theta_0} + \varepsilon W_{\theta_1} + \cdots + \varepsilon^p W_{\theta_p}) \quad (11)$$

Provided the initial conditions $x(0) = A$ and $x'(0) = B$, they may be enforced entirely at the zeroth order by setting $x_n(0) = 0$ and $x'_n(0) = 0$ for $n > 0$ in the *leading-order initial condition* approach. Alternatively, the initial conditions may be distributed uniformly across perturbation orders in the *uniform initial condition* approach by setting

$$x_n(0) = \frac{A}{\sum_{k=0}^p \varepsilon^k}, \quad x'_n(0) = \frac{B}{\sum_{k=0}^p \varepsilon^k}.$$

3.3.4. Initial condition handling in PTL-PINNs

In Fig. 2, we compare the *leading-order initial condition* and *uniform initial condition* by solving with PTL-PINNs the unforced overdamped oscillator for various initial conditions using fifteen correction terms for a non-linearity of $0.5x^3$ and damping $\zeta = 10$.

An overdamped regime was chosen because, unlike the undamped and underdamped regimes, it does not exhibit resonance or near-resonance behavior, as its homogeneous solution is purely exponential. Despite this, the perturbation series will still diverge for large initial conditions. This limitation is inherent to perturbation theory and arises because the perturbation terms are multiplied and raised to high powers throughout the perturbation system. It can be mitigated by adopting alternative approaches to the treatment of initial conditions.

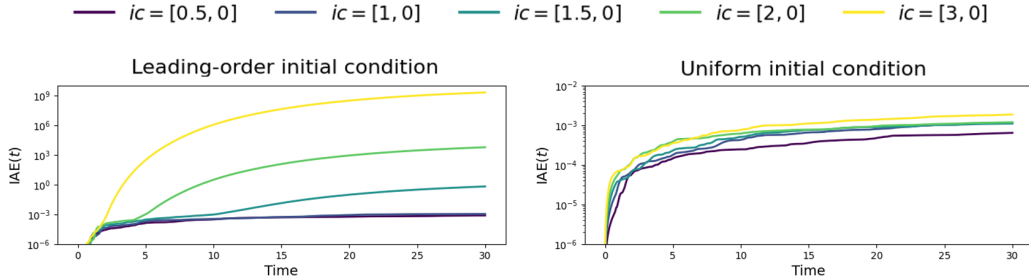


Figure 2: Comparison of initial-condition handling strategies for the unforced overdamped oscillator ($\zeta = 10$). PTL-PINN solutions are shown using 15 perturbation corrections for a cubic nonlinearity $0.5x^3$. The *leading-order* approach enforces the full initial condition at zeroth-order only, while the *uniform* approach distributes the initial condition evenly across all perturbation orders. Results are shown for increasing initial positions $x(0) \in \{0.5, 1, 1.5, 2, 3\}$ with zero initial velocity.

The *uniform* approach leads to low errors for larger initial conditions. By distributing the initial condition evenly across all orders, the starting amplitude of lower-order corrections is kept within a manageable range. This

delays the onset of a blow-up, which is largely determined by the amplitude of x_0 . However, the *leading-order* approach has various advantages. It produces a truncatable series, where the contribution of each order is independent of the initial conditions. In contrast, the *uniform* approach cannot be truncated without losing information, since satisfying the initial condition requires including all corrections. Due to its interpretability, the results shown throughout this manuscript have considered the *leading-order* approach.

3.4. Importance of PTL-PINN pretraining

In one-shot transfer learning, we obtain the solution by taking a linear combination of the latent representation as described in equation 11. As such, the perturbation-guided pretraining stage is crucial to ensure a meaningful latent representation is learned. In this section, we demonstrate the importance of the pretraining stage. Fig. 3 shows the results of using nine corrections to solve various forced overdamped oscillators with models trained on undamped (a)-(d), underdamped (e)-(h) and overdamped (i)-(l) linear equations.

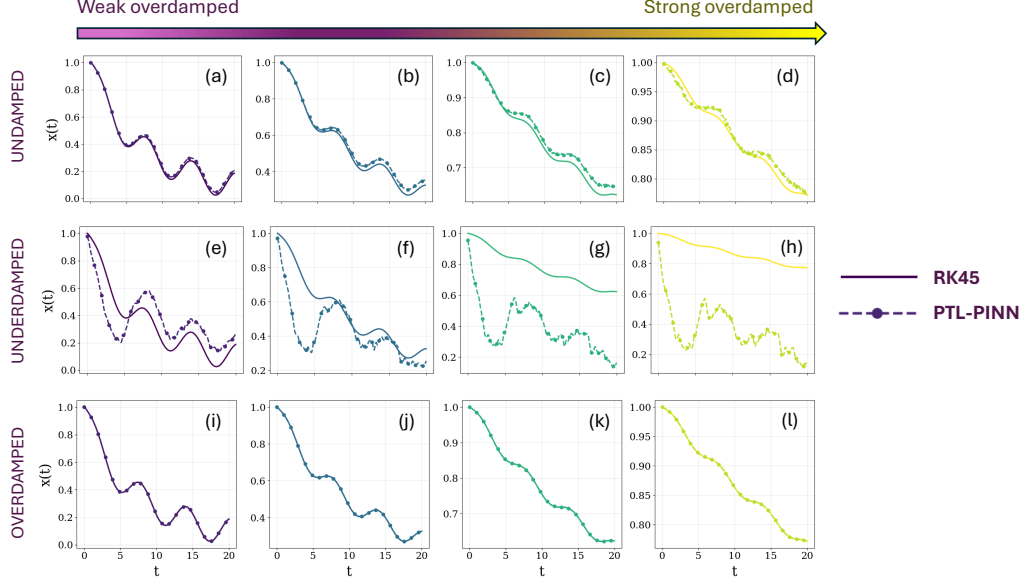


Figure 3: Comparison of the solution of various nonlinear overdamped oscillators using the baseline RK45 and PTL-PINN undamped, underdamped and overdamped models. Each row corresponds to a fixed PTL-PINN model, while each column represents a different damping value from $\{5, 10, 30, 60\}$. The PTL-PINN overdamped model has greatest accuracy.

Fig. 3 (i)-(l) shows that the PTL-PINN trained on overdamped equations accurately reconstructs the nonlinear RK45 reference solution. In contrast, the undamped and underdamped models are unable to reconstruct the overdamped solution, which highlights the need to train a model for each regime.

Besides the parameters considered for the pretraining equations, the number of pretraining equations (heads) directly impacts the results. Fig. 4 shows how well different PTL-PINN models can recover Lindstedt–Poincaré frequency series for the unforced undamped oscillator, described in equation 6. The left plots in Fig. 4 show the absolute relative frequency contribution, which quantifies the impact of each frequency correction on the series, as more perturbation corrections are considered. The right plots show the frequency series MAE, where RK45 provides reference frequency, computed from the peak-peak distance over the short interval $[0, 4\pi]$. To maximize accuracy, we consider with absolute and relative tolerances of 10^{-10} for RK45.

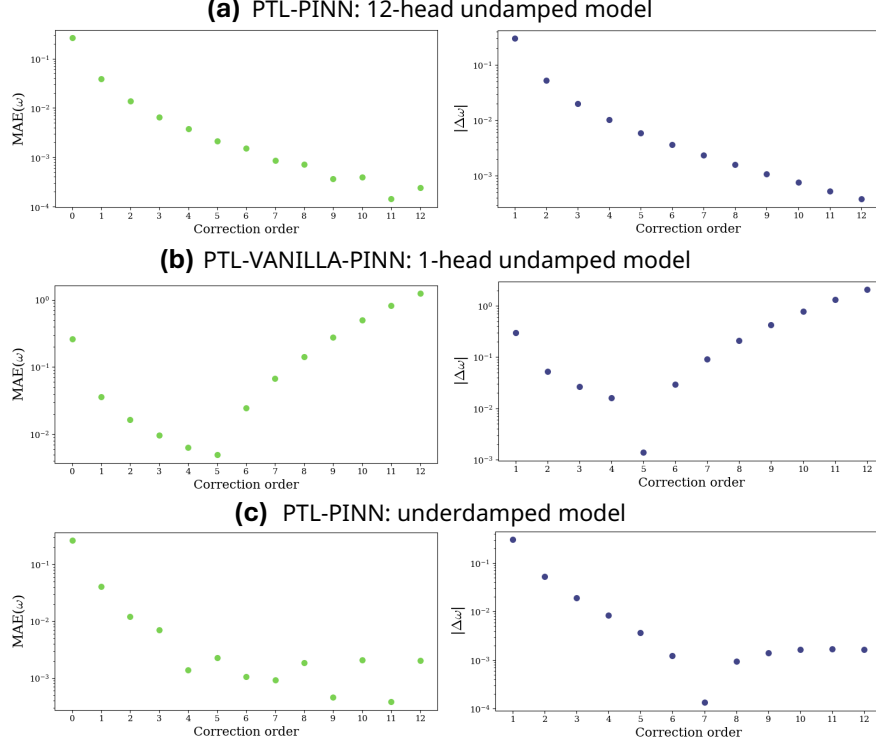


Figure 4: Lindstedt–Poincaré frequency convergence for the undamped and unforced oscillator with $x(0) = 1$ and $x'(0) = 0$ using an undamped (twelve-headed), underdamped and vanilla (one-headed) model, showing absolute relative frequency contribution (right) and frequency series MAE (left)

Fig. 4 (a) shows the results for a twelve-headed undamped PTL-PINN model, which achieved the lowest MAE(ω) and $|\Delta\omega|$, both decreasing steadily with each order. Fig. 4 (b) presents the results for a one-headed undamped PTL-PINN. Trained on a single equation, the model’s latent representation is not expressive enough to capture the underlying dynamics, leading to frequency divergence beyond the fifth correction. Fig. 4 (c) shows the results for a PTL-PINN trained on twelve underdamped systems. As these pretraining equations differ from the target dynamics, the resulting latent representation lacks generalizability, leading to frequency divergence beyond the seventh correction.

We notice there is a correlation between the relative frequency and its

absolute error. When the relative frequency does not diminish as we consider more corrections, we find that adding more corrections does not necessarily imply a more accurate perturbation series with lower absolute error. As a result, a steadily diminishing relative frequency provides a stopping criterion for our algorithm. This allows divergence to be identified when solving our perturbation system and early truncation.

To further benchmark the results of our PTL-PINN undamped model, we solve the Lindstedt–Poincaré perturbative system of the unforced undamped oscillator with RK45 for nonlinearity $0.5x^3$ and initial conditions $x(0) = 1$ $\dot{x}(0) = 0$. Fig. 5 compares the solution of the first, fifth and eighth-order corrections from RK45, a PTL-PINN twelve-headed model and a PTL-PINN one-headed model.

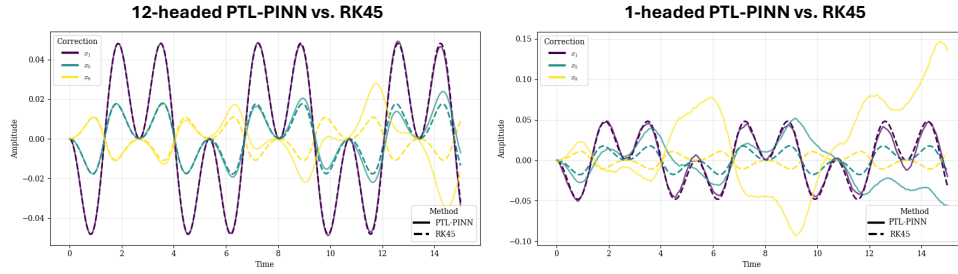


Figure 5: Comparison of the first, fifth and eighth-order corrections when using RK45 and a twelve-headed and one-headed PTL-PINN model.

These results show that the twelve-headed PTL-PINN has excellent agreement with the RK45 solution for the first and fifth-order corrections, with a larger deviation observed for the eighth-order correction. Conversely, a one-headed PTL-PINN model is only able to find the first-order correction without significant total error. Fig. 5 demonstrates that PTL-PINN accuracy is highly dependent on the number of pretraining equations (heads), being limited by the expressiveness of the latent representation. Moreover, these results show the challenge PTL-PINNs face in generalizing to higher-order corrections, especially for models trained on a small number of equations or equations unrelated to the system being solved.

4. Accuracy Results on Nonlinear Systems

4.1. Numerical solver baselines

We compare the accuracy of PTL-PINNs against the RK45 solution of the nonlinear equation, since it is a widely used explicit method with fourth-order accuracy [42]. To further benchmark accuracy, we also solve the Lindstedt–Poincaré and standard perturbation systems using RK45 solely to assess how accurately the PTL-PINN reproduces the perturbation-theory solutions across various orders of perturbation. We evaluate accuracy using the mean absolute error (MAE) and the integrated absolute error (IAE):

$$\text{MAE}(x_{\text{approx}}) = \frac{1}{N} \sum_{i=0}^{N-1} |x_{\text{approx}}(i) - x_{\text{reference}}(i)| \quad (12)$$

$$\text{IAE}(t) = \int_0^t |x_{\text{approx}}(m) - x_{\text{reference}}(m)| \, dm \quad (13)$$

To assess computational efficiency, we also consider Radau, an implicit Runge-Kutta method particularly well-suited for stiff systems [43]. For PDEs, we first discretize the spatial domain and then apply the time integration method RK45, which operates on the resulting system of ordinary differential equations obtained from the finite-difference discretization.

The numerical solvers considered, RK45 and Radau, have adaptive time-stepping. For a given relative and absolute tolerance, these solvers adjust the time step to ensure the local error estimate remains less than the sum of the absolute tolerance plus the relative tolerance multiplied by the absolute value of the solution. As such, for any accuracy baselines, we ensure high accuracy by setting absolute and relative tolerances of 10^{-10} . For our computational efficiency benchmarks, we consider tolerances of 10^{-3} for RK45 and Radau, which we demonstrate to be comparable to the accuracy achieved by PTL-PINNs.

4.2. ODEs: Damped and Undamped Oscillator

The expected structure of the perturbative solution for the nonlinear oscillator depends on the damping regime. To ensure that a meaningful latent representation is learned, we train a separate PTL-PINN model for each of the undamped, underdamped and overdamped cases, as described in Section Appendix B.

PINNs often struggle with spectral bias in training. To mitigate this, when solving the undamped and underdamped oscillator, we use Fourier feature embeddings [44], which map the inputs through sinusoidal functions with various uniformly sampled frequencies. Furthermore, we use sinusoidal activation functions for the first two layers of the undamped and underdamped models [45] and for the first layer of the overdamped model.

In Fig. 6, we show that PTL-PINNs using five corrections are able to recover, with MAE of order 10^{-2} or less, the unforced undamped oscillator using the Lindstedt–Poincaré method and the underdamped ($\zeta = 0.5$) and overdamped ($\zeta = 5$) oscillators forced by $\cos(t)$ using the standard perturbation method for the damped cases.

We consider initial conditions $x(0) = 1, \dot{x} = 0$ and a polynomial nonlinearity of $-x^3 + x^5$ with the coefficient $\varepsilon = 0.5$, which corresponds to a quintic approximation of the nonlinearity $\sin(x)$. PTL-PINNs naturally extend to nonlinearities of higher polynomial order within the same framework.

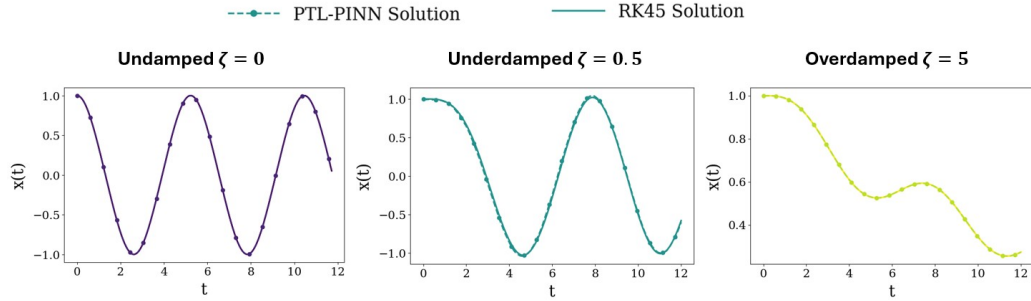


Figure 6: PTL-PINN vs. baseline RK45 solutions for the unforced and undamped oscillator and the forced underdamped ($\zeta = 0.5$) and overdamped ($\zeta = 5$) oscillators under external forcing $\cos(t)$. PTL-PINN solutions are shown to be accurate using Lindstedt–Poincaré method for the undamped oscillator and the standard perturbation method for the damped oscillators.

4.3. Coupled ODEs: Predator–prey Models

The Lotka–Volterra system describes the nonlinear interaction between two species in a predator–prey system. Their normalized version is given by:

$$\begin{aligned}\dot{x} &= x - xy, \\ \dot{y} &= -\alpha y + \alpha xy,\end{aligned}\tag{14}$$

where $x(t)$ and $y(t)$ represent the prey and predator populations and $\alpha > 0$. This system admits a non-trivial steady state $(x^*, y^*) = (1, 1)$. We apply

PTL-PINNs in combination with the Lindstedt–Poincaré method to approximate solutions of the Lotka–Volterra system in a neighborhood of the non-trivial steady state.

Following Grozdanovski weakly nonlinear formulation of the Lotka–Volterra system [46], we introduce perturbative variables ξ and η defined by $x(t) = 1 + \varepsilon\xi(t)$ and $y(t) = 1 + \eta(t)$. Then, we define their perturbative series as $\xi = \xi_0 + \xi_1\varepsilon + \dots$ and $\eta = \eta_0 + \eta_1\varepsilon + \dots$ and implement the Lindstedt–Poincaré method numerically similarly to how we approached the undamped oscillator. More details on the derivation of the general n -th order differential equation and frequency correction can be found in the corresponding methodological subsection present in Appendix A.2.

Because the functional form of the perturbative solutions resembles that of the undamped oscillator, we reuse the PTL-PINN model trained for the undamped oscillator to solve the Lotka–Volterra for $\alpha = 0.2$ and $\varepsilon = 0.5$. We consider initial conditions $x(0) = 1.59$ and $y(0) = 0.95$ in Fig. 7 (a) and $x(0) = 1.75$ and $y(0) = 0.85$ in Fig. 7 (b).

Fig. 7 shows that PTL-PINNs combined with the Lindstedt–Poincaré method can accurately recover the periodic orbits around the non-trivial critical point $(x^*, y^*) = (1, 1)$.

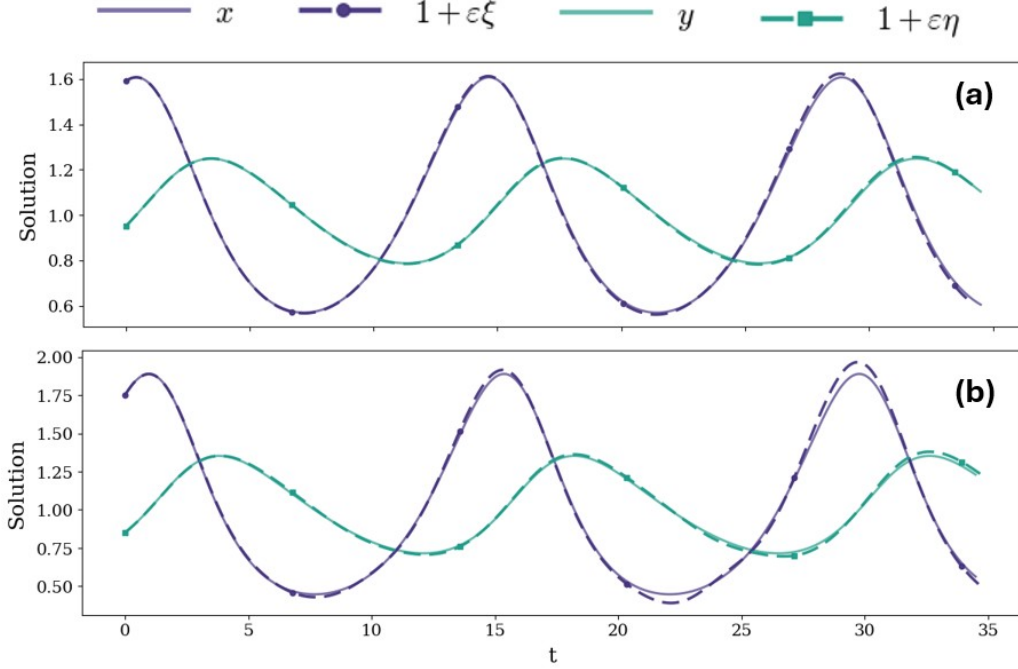


Figure 7: PTL-PINN (dashed line) vs. RK45 baseline (continuous line) solutions of the equilibrium-centered Lotka–Volterra system for $\alpha = 0.2$ and $\epsilon = 0.5$. The same latent representation trained on the undamped oscillator is reused to recover periodic orbits around the non-trivial equilibrium $(1, 1)$ using the Lindstedt–Poincaré method. Panels (a) and (b) correspond to increasing deviations from equilibrium with initial conditions $x(0) = 1.59$, $y(0) = 0.95$ for (a) and $x(0) = 1.75$, $y(0) = 0.85$ for (b).

Because the Lindstedt–Poincaré expansion is constructed around this equilibrium, we expect its accuracy to deteriorate as the initial condition moves farther from this point. In fact, PTL-PINN achieves MAE of 8×10^{-3} for $\xi(0) = 1.59$ and $\eta(0) = 0.95$. However, for $\xi(0) = 1.75$ and $\eta(0) = 0.85$, it achieves a lower MAE of 3×10^{-2} as this point further is from $(1, 1)$. PTL-PINNs can also be applied in regimes of weak coupling between species, where the cross-species interaction term (xy) remains small relative to each population’s self-dynamics.

4.4. PDEs: KPP–Fisher and Wave Equation

Reaction-diffusion and wave phenomena appear across a wide range of systems and can be modeled by the KPP–Fisher and Wave equations. The KPP–Fisher equation models population growth and wave propagation [47,

48]. To apply the PTL-PINN framework, we reformulate the systems in their weakly nonlinear form in accordance with PTL-PINNs' general formulation presented in equation 1.

Following this formulation, the KPP–Fisher weakly nonlinear form is given by equation 15, where the linear operator is D is the diffusion coefficient, $\hat{D} = \frac{\partial}{\partial t} - D \frac{\partial^2}{\partial x^2}$ is the linear differential operator and $\mathcal{N} = -u(1 - u)$ is the nonlinear operator:

$$\frac{\partial u}{\partial t} - D \frac{\partial^2 u}{\partial x^2} - \varepsilon u(1 - u) = f(x, t) \quad (15)$$

Similarly, the Wave equation weakly nonlinear formulation is given by equation 16, where the linear wave speed is c , the linear operator is $\hat{D} = \frac{\partial^2}{\partial t^2} - c^2 \frac{\partial^2}{\partial x^2}$ and the nonlinear operator is $\mathcal{N} = u^q$.

and its weakly nonlinear formulation is:

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} + \varepsilon u^q = f(x, t), \quad (16)$$

We solve these equations for $x \in (0, 2)$, $t \in (0, 5)$ and $\varepsilon = 0.5$. The training equations are provided in Appendix B.2 and are guided by perturbation theory as discussed earlier. Fig. 8 shows PTL-PINN solutions of the KPP–Fisher for Dirichlet boundary and initial conditions of varying complexity and frequency. In Fig. 8 (a), we solve for the initial condition $\sin(2\pi x/L)$ with diffusion coefficient $D = 0.1$. Fig. 8 (b), the solution was obtained for $D = 0.01$ and the initial condition $\sin^2(3\pi x/L)$, while Fig. 8(c) was computed using $D = 0.01$ with the initial condition $\sin^2(5\pi x/L)$. As the complexity of the initial condition increases from (a) to (c), the corresponding MAE values increase, being 4.6×10^{-4} , 6.3×10^{-3} and 4.0×10^{-2} for (a), (b) and (c), respectively.

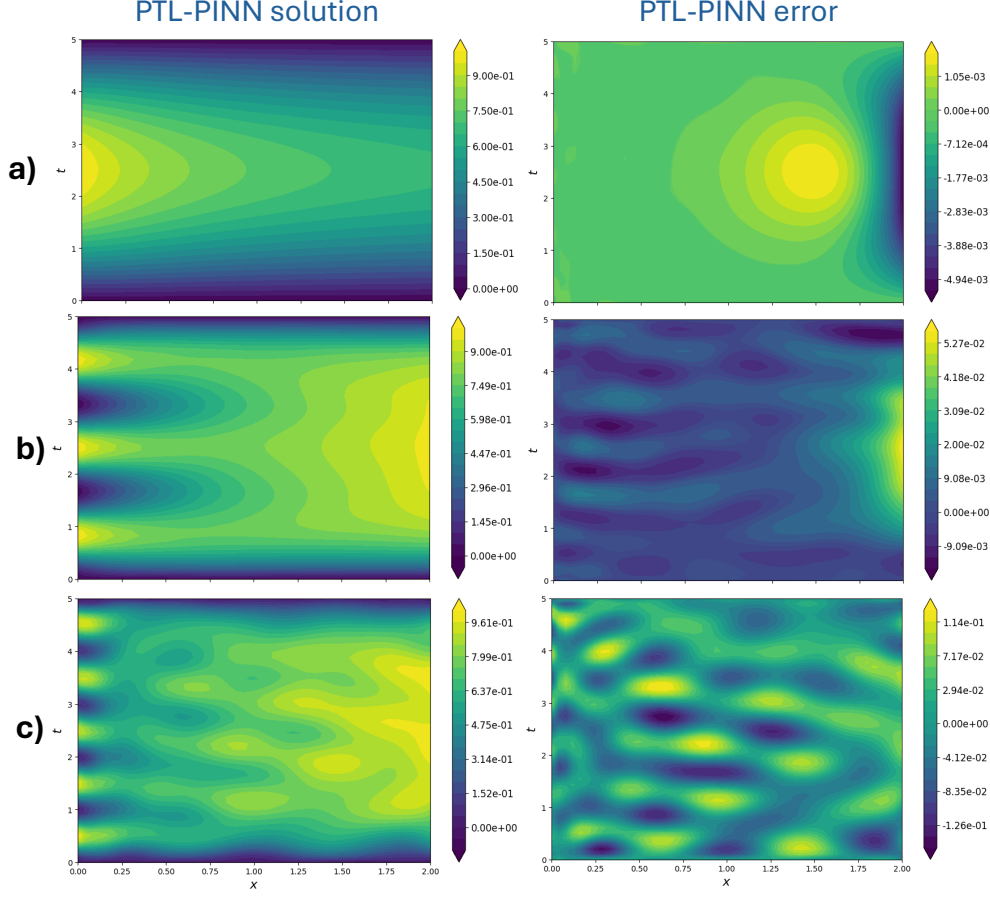


Figure 8: PTL-PINN solutions (left panel) and errors (right panels) for the weakly nonlinear KPP–Fisher equation for different initial conditions. The initial conditions are progressively more complex from (a)-(c), taking higher frequency and/or more harmonics. Consequently, the error increases from (a)-(c) as it becomes harder for PTL-PINN to generalize. MAE values are 4.6×10^{-4} , 6.3×10^{-3} and 4.0×10^{-2} for (a), (b) and (c), respectively.

The same is replicated for the nonlinear Wave equation with nonlinearity of power $q = 3$. Fig. 9 shows the PTL-PINN solution for the Wave equation, considering Dirichlet boundaries, linear wave velocity of $c = 1$ and initial conditions of varying frequency. In Fig. 9(a), we consider the initial condition $\sin(2\pi x/L)$. Fig. 9(b) corresponds to the initial condition $\sin(3\pi x/L)$. As the frequency of the initial condition increases from (a) to (b), the corre-

sponding MAE values increase, being 2.9×10^{-4} and 7.9×10^{-3} for (a) and (b), respectively.

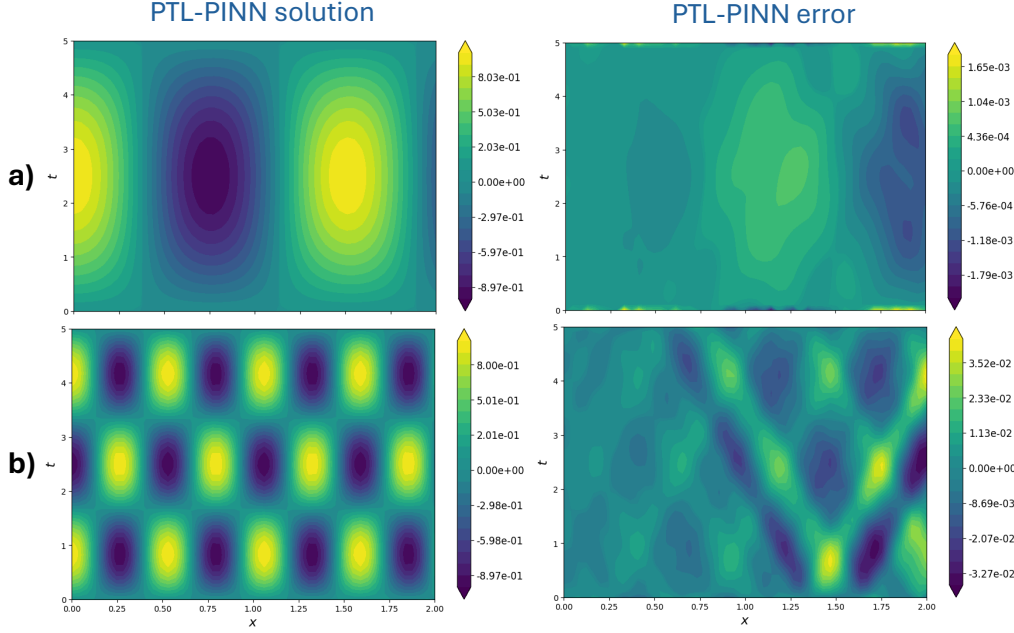


Figure 9: PTL-PINN solutions (left panel) and errors (right panels) for the weakly nonlinear Wave equation with Dirichlet boundaries, linear wave velocity $c = 1$ and equation for different initial conditions. Panel (a) corresponds to the initial condition $\sin(2\pi x/L)$ and panel (b) corresponds to the Wave equation with an initial condition $\sin(3\pi x/L)$. As the frequency of the initial condition increases from (a) to (b), the corresponding MAE values increase, indicating higher error for the higher frequency condition.

In this subsection, we demonstrated the success of PTL-PINNs on benchmark PDEs and their ability to scale to more complex initial conditions. This framework can be naturally scaled to more complex PDEs and higher frequencies by adjusting the model and training equations. This can be extended to any PDE, first requiring its weakly nonlinear formulation. Various PDEs, such as the Navier-Stokes, have been solved with perturbation theory [49].

5. Computational Efficiency

PTL-PINNs solve each linear equation in the perturbative system by multiplying a precomputed latent representation $H(t, x)$ by a matrix W_θ , which is obtained via the closed-form expression in equation 10. The computation of W_θ involves matrix multiplications and the inverted matrix M^{-1} . As discussed in Section 3.3.2, M^{-1} is independent of the initial/boundary conditions, forcing function or nonlinearity. Consequently, the same matrix M^{-1} can be reused across all perturbation orders in the same system, since only the initial/boundary conditions and forcing terms vary. Moreover, it can be reused between nonlinear equations, where only the forcing, initial conditions or nonlinearity varies. If M^{-1} is unavailable, the solution is found with a time complexity equivalent to that of a matrix inversion. If it is, then the solution is even faster with the time complexity of matrix-vector multiplications.

Table 1 compares the solve time of the unforced case of the PDEs and ODEs previously discussed using: PTL-PINNs (reusing and inverting M), classical solvers (RK45 and Radau) and regular transfer learning (Regular-TL). In Regular-TL, we trained a Multi-Headed-PINN on various versions of the nonlinear oscillator that we solve. We freeze the shared backbone of the network and find the final layer W with gradient-based optimization. The computational time expresses how long it took for Regular-TL to reach $\text{MAE} < 2.5 \times 10^{-2}$, when compared to a low absolute tolerance RK45 solution. Our results show that Regular-TL is unreliable, taking many orders of magnitude longer than other solvers, only to find a less accurate solution. Conversely, the PTL-PINN solutions can be up to one order of magnitude faster than classical solvers.

Equation	Time (milliseconds)				
	RK45	Radau	Regular TL*	PTL-PINN-invert	PTL-PINN-no-invert
Undamped	28.9	11.1	2.4×10^4	70.2	4.8
Underdamped ($\zeta = 0.4$)	4.7	1.2	1.4×10^5	19.1	1.1
Underdamped ($\zeta = 0.6$)	3.7	1.4	1.8×10^3	14.0	1.1
Overdamped ($\zeta = 10$)	3.6	6.4	3.23×10^4	21.7	1.1
Overdamped ($\zeta = 30$)	2.0	22.8	7.75×10^5	14.1	1.1
Fisher-KPP	47.7	6.2	6.5×10^3	311.1	15.0
Wave equation	64.4	263.8	8.3×10^3	396.5	18.3
Lotka-Volterra ($\alpha = 0.2$)	1.3	6.0	3.15×10^5	62.9	6.3

Table 1: Solve time for PTL-PINNs (inverting and not inverting M), Regular TL, Radau and RK45 for various equations. The asterisk (*) denotes that Regular TL times were obtained when the solution matches the RK45 numerical solution with a tolerance of 2.5×10^{-2} , which is a substantially larger error than that obtained with the other solvers.

The equations were solved for $t \in (0, 10)$, $\varepsilon = 0.5$ and without forcing. We use a monotonic clock with the highest available resolution to measure the time and average eight hundred iterations of our numerical solvers and PTL-PINN results. For PTL-PINNs, we exclude the one-time cost of computing $H(t)$ and derivatives since they can be reused. As we will later justify in this section, we consider five correction terms, $N = 150$ inference points for ODEs and $N = 50 \times 50$ points for PDEs. As discussed in Section 4.1, we set the tolerances of RK45 and Radau to 10^{-3} for fair comparison of computational efficiency. Table 2 shows the MAE achieved for the solvers presented in Table 1, where the reference is RK45 with tolerance of 10^{-10} . The values of RK45 and Radau consider the same tolerance as in Table 1, which is 10^{-3} .

Equation	Accuracy (MAE)			
	RK45	Radau	Regular TL*	PTL-PINN
Undamped	1.1×10^{-2}	7.6×10^{-5}	2.5×10^{-2}	2.2×10^{-3}
Underdamped ($\zeta = 0.4$)	6.9×10^{-4}	4.1×10^{-5}	2.5×10^{-2}	4.8×10^{-3}
Underdamped ($\zeta = 0.6$)	9.1×10^{-4}	3.4×10^{-5}	2.5×10^{-2}	8.1×10^{-3}
Overdamped ($\zeta = 10$)	2.5×10^{-5}	2.9×10^{-5}	2.5×10^{-2}	1.5×10^{-4}
Overdamped ($\zeta = 30$)	6.0×10^{-6}	8.2×10^{-6}	2.5×10^{-2}	4.2×10^{-4}
Fisher-KPP	4.3×10^{-4}	2.0×10^{-5}	2.5×10^{-2}	4.6×10^{-4}
Wave equation	1.1×10^{-3}	5.6×10^{-4}	2.5×10^{-2}	2.9×10^{-4}
Lotka-Volterra ($\alpha = 0.2$)	2.5×10^{-3}	2.5×10^{-3}	2.5×10^{-2}	5.3×10^{-3}

Table 2: MAE of PTL-PINNs, Regular TL, Radau and RK45 for various equations. PTL-PINNs models achieve accuracy of order 10^{-3} and 10^{-4} , which is comparable to RK45 and Radau.

To justify our choice of five correction terms, we examine how the solve time and accuracy vary with the number of corrections when using PTL-PINN-no-invert for the previously considered ODEs, this time including forcing for the damped ODEs. The results are shown in Fig. 10. The left plots show the average computational time over twenty-five iterations for each order, revealing a nearly linear increase with the perturbation order. The right plots show that the logarithm of the mean absolute error scales linearly with the number of perturbation terms. This is to be expected since the n th-order terms are scaled by ε^n . In these right plots, we show the theoretical expected accuracy in a dashed line, which was obtained by solving the perturbative system using RK45 with tolerances of 10^{-10} . We observe that PTL-PINN accuracy across orders follows the theoretical RK45 accuracy across perturbation order. This demonstrates that PTL-PINNs successfully solve the different equations in the perturbative system with high accuracy.

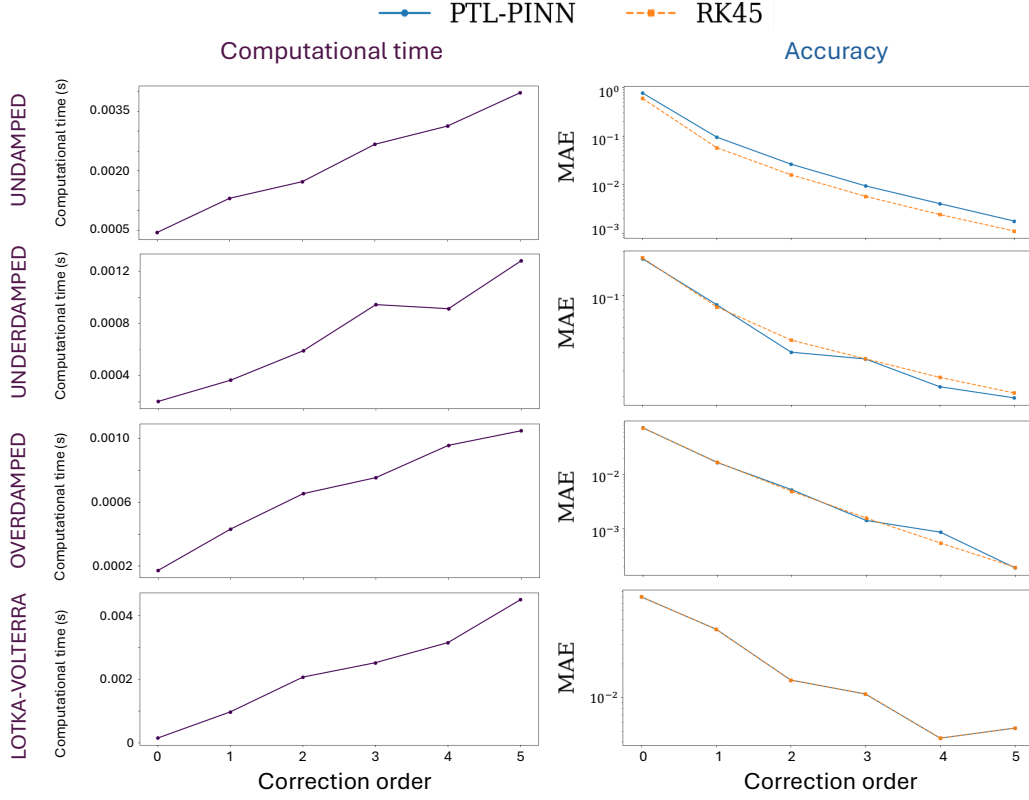


Figure 10: Computational time (left panel) and accuracy (right panel) of PTL-PINNs across correction orders for different ODEs and regimes. The computational time increases linearly exponentially with the number of correction terms.

Moreover, in Table 1 and 2, we considered $N = 150$ points for ODEs and $N = 50 \times 50$ for PDEs with PTL-PINNs, which for the time interval considered in ODEs, $t \in (0, 10)$, returns a solution evaluated uniformly at points in this interval with time step ≈ 0.067 . The computational time of RK45 and Radau is not significantly influenced by the number of points chosen, since it depends instead on the absolute and relative tolerances considered. However, the more points are considered, the longer PTL-PINNs will take to solve the equation since $H(x, t)$ and the other matrices multiplied will be larger. In Fig. 11, we show that the PTL-PINNs accuracy for ODEs is not affected by the number of points chosen. However, it is important to note that the runtime of this method scales directly with N , so N must be chosen to balance computational cost and the desired resolution of the final solution.

The ODEs considered are unforced and the same as the ones detailed for Table 2.

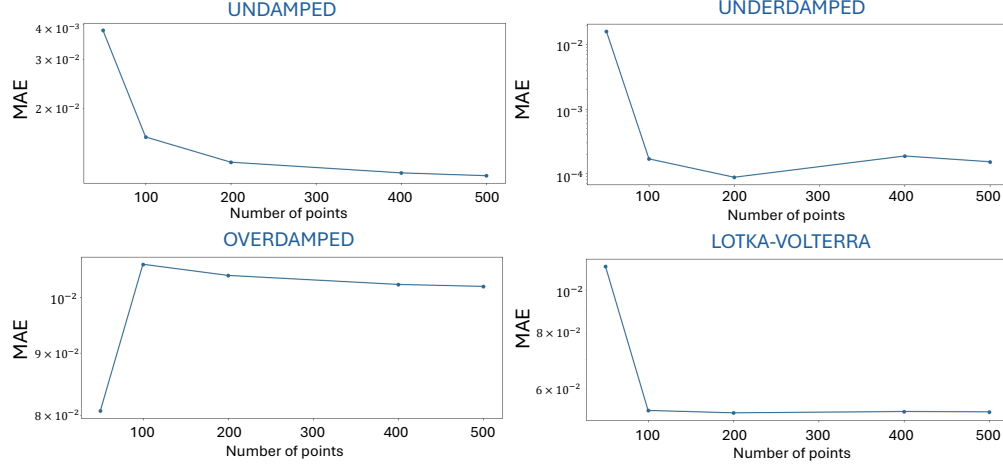


Figure 11: Mean absolute error of the PTL-PINN solution vs. number of inference points N for various ODEs. The inference points take values in $\{0, 100, 200, 400, 500\}$. Apart from $N = 50$, the mean absolute error remains close to constant for different N .

6. Perturbation Limitations and Solutions

6.1. Standard Perturbation Method Failure Modes

In this subsection, we examine the factors that influence the accuracy of perturbation theory approximations. Fig. 12 shows the integrated absolute error, $\text{IAE}(t)$, of the PTL-PINN solutions obtained using the standard perturbation method for both a forced underdamped oscillator and an unforced undamped oscillator, as increasingly higher-order corrections are included. At longer time horizons, we observe that perturbative approximations including additional correction terms can exhibit larger errors.

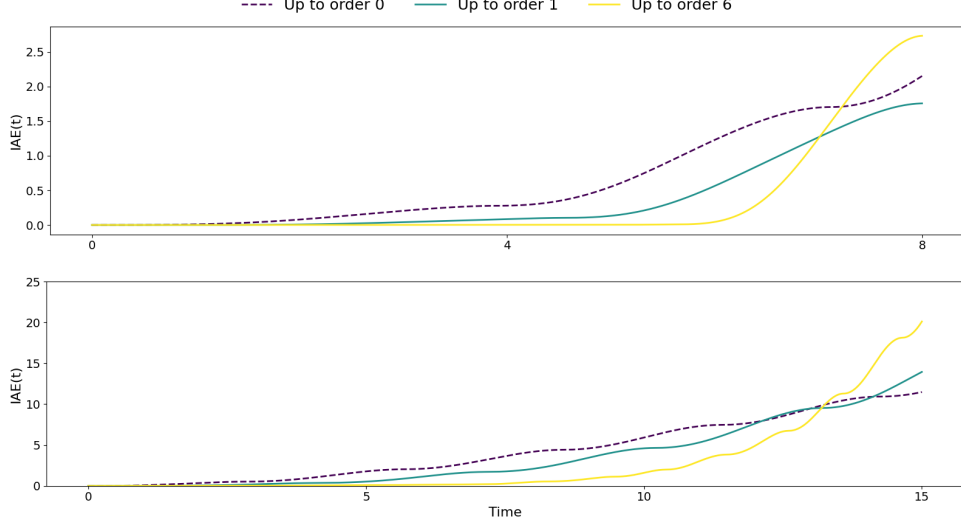


Figure 12: Integrated absolute error $IAE(t)$ of PTL-PINN solutions obtained using standard perturbation theory for (a) a forced underdamped oscillator and (b) an unforced undamped oscillator, as progressively higher-order perturbation corrections are included. Results are shown for the zeroth-order solution, the first-order correction and the sixth-order correction. In both cases, higher-order perturbative expansions can lead to increased long-time error due to resonant or near-resonant effects inherent to the standard perturbation method.

Naively, one might expect that adding more correction terms to the perturbative series would always improve accuracy. However, the standard perturbation method for the undamped oscillator introduces terms of the form of $t^n \sin(t)$, which correspond to the resonant, non-physical terms discussed in Section 3.2. These terms artificially increase the oscillator’s energy and grow with time, which explains the increased error.

In the underdamped case, there is no pure resonance since the homogeneous solution is not purely oscillatory, taking the functional form of $e^{-\omega_0 \zeta t} \cos(\omega_0 \sqrt{1 - \zeta^2} t + \phi)$. However, the forcing terms in the perturbative system can have frequencies close to the homogeneous solution, leading to near-resonance effects that degrade accuracy. Such near-resonant behavior arises only in specific regions of the underdamped regime.

Fig. 12 (a) illustrates this behavior for an oscillator with very low damping ($\zeta = 0.25$) under external forcing. Due to small damping, the frequency

of the harmonics in the homogeneous solution, which is $\sqrt{1 - \zeta^2}$, approaches unity, which coincides with the frequency of the forcing at zeroth-order. Moreover, the exponential decay term, $e^{-\omega_0 \zeta t}$, does not dominate the dynamics over the time interval considered. When the forcing is removed or the damping is increased, the standard perturbation method achieves high accuracy for the same parameters.

Nonetheless, higher-order corrections can improve accuracy in the standard perturbation method over short time scales and for sufficiently small initial conditions, since the non-physical terms initially have a small amplitude. The impact of these terms further diminishes as the nonlinearity coefficient ε decreases, because ε scales the magnitude of the perturbative corrections and controls how close the nonlinear frequency remains to the linear one.

These observations suggest that standard perturbation theory can remain effective when combined with early truncation of the series. At the same time, they motivate the use of alternative perturbation approaches, such as the Lindstedt–Poincaré method, when long-time accuracy or oscillatory fidelity is required.

In Fig. 13, we compare the PTL-PINN solution of the unforced undamped oscillator obtained using the standard perturbation and Lindstedt–Poincaré methods against the RK45 numerical reference. We observe that the Lindstedt–Poincaré formulation successfully removes the resonant terms of the form $t^n \sin(t)$, which were responsible for the monotonic growth of the integrated absolute error with time in Fig. 12 (b).

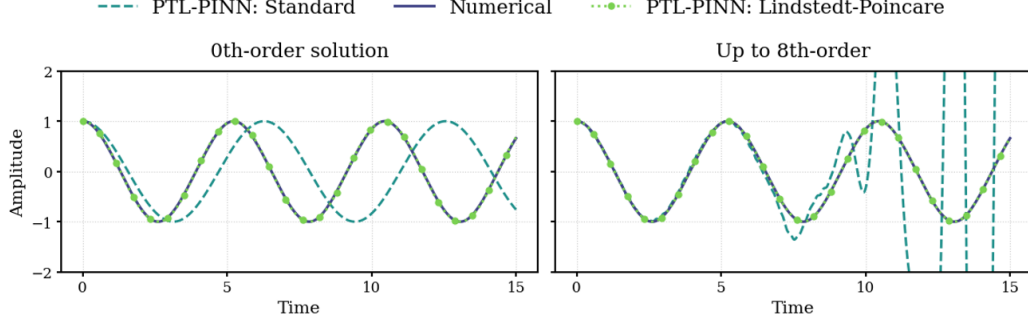


Figure 13: PTL-PINN solutions of the unforced and undamped oscillator against an RK45 baseline. The PTL-PINN employing the Lindstedt–Poincaré method accurately recovers the correct solution. The right panel shows PTL-PINN results using only the zeroth-order correction, which has lower error than the left panel results, which consider eight correction terms.

6.2. Lindstedt–Poincaré method with forcing

Throughout this manuscript, we have primarily considered unforced oscillators when applying PTL-PINNs in combination with the Lindstedt–Poincaré method. In this subsection, we show that in the presence of external forcing, the zeroth-order Lindstedt–Poincaré solution requires an initial approximation, since the frequency correction series has not yet been determined. As a result, for forced systems, improved accuracy can be achieved by applying the method in multiple passes.

In the first pass, at zeroth-order, we approximate the forcing-phase using $t/\omega \approx t/\omega_0$. Using this approximation, the perturbation system is solved within the PTL-PINN framework, yielding a first-pass frequency estimate, $\omega^{(1)}$. To improve accuracy, the Lindstedt–Poincaré procedure can be rerun, but with the zeroth-order approximation $t/\omega \approx t/\omega^{(1)}$.

In Fig. 14, we illustrate this by comparing the MAE of the first-pass and second-pass solutions of the Lindstedt–Poincaré method for a nonlinearity $0.8x^3$, external forcing $\cos(t)$ and initial conditions $x(0) = 1$ and $\dot{x}(0) = 0$. The zeroth-order forcing term in the first pass is $\cos(6t/\omega_0) = \cos(6t)$, while in the second pass it becomes $\cos(6t/\omega^{(1)}) \approx \cos(4.74t)$.

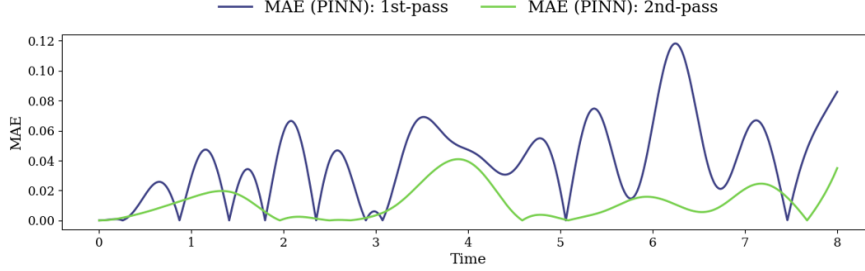


Figure 14: PTL-PINN solution mean absolute error for a forced nonlinear oscillator considering the Lindstedt–Poincaré method applied once (first pass) and twice (second pass). In the first pass, the forcing-phase rescaling uses the linear frequency ω_0 , yielding a first estimate $\omega^{(1)}$. In the second pass, the zeroth-order approximation is updated using $\omega^{(1)}$, leading to a refined frequency estimate $\omega^{(2)}$ and improved accuracy.

7. Conclusion and future work

We presented PTL-PINN, a perturbation-guided transfer learning framework, which combines perturbation theory and PINNs. In various benchmark problems, we have successfully demonstrated that PTL-PINNs can accurately and efficiently solve perturbative systems that approximate weakly nonlinear ODEs and PDEs. By using one-shot transfer learning, PTL-PINNs are orders of magnitude faster than gradient-based transfer learning and of comparable speed to RK45 and Radau. This work addresses the fundamental challenges of PTL-PINNs: (1) learning a meaningful latent representation and (2) ensuring the linear system derived from perturbation theory is an accurate approximation.

To address (1), we use the general expected solution of a perturbative system to guide our pretraining. To further improve our results, larger models could be trained on more challenging equations. In addition, regularization strategies can be employed to improve transferability [50, 51]. To address (2), the perturbation series may need early truncation, alternative perturbation methods should be considered or the method should be avoided altogether for certain parameters. We implemented the Lindstedt–Poincaré method for the undamped oscillator and the equilibrium-centered Lotka–Volterra system. Several analytical variants of the Lindstedt–Poincaré method [52, 53] and other perturbation methods, such as the Method of Multiple Scales, will be explored in future work.

This work lays the groundwork for extending PTL-PINNs to a broader class of nonlinear systems. Extending this framework to higher-dimensional

and more complex cases, such as the three-body problem and the Navier-Stokes, which have been studied with perturbation theory [54, 49], will be the focus of future work. PTL-PINNs delivers foundational models that can compute solutions without retraining and with the time complexity of matrix-vector multiplication, broadening the applicability of PINNs to real-life scenarios where computational cost is a limitation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The code has been made publicly available on <https://github.com/esemsc-dda24/PTL-PINNs>.

Declaration of AI-assisted technologies

During the preparation of this work, the authors used ChatGPT (OpenAI GPT-5) to assist with minor language refinement. The tool was not used to generate scientific ideas, analyses, results or interpretations. After using this tool, the authors critically reviewed and verified any suggestions. The authors take full responsibility for the content of the published article.

CRedit authorship contribution statement

Duarte Alexandrino: Conceptualization, Methodology, Software, Writing - original draft, Writing - review & editing. **Ben Moseley:** Supervision, Writing - review & editing. **Pavlos Protopapas:** Conceptualization, Supervision, Writing - review & editing

References

- [1] S. Larsson, V. Thomée, Partial differential equations with numerical methods, Springer, 2003.

- [2] J. R. Martins, A. Ning, Engineering design optimization, Cambridge University Press, 2021.
- [3] A. Jameson, Aerodynamic design via control theory, *Journal of scientific computing* 3 (3) (1988) 233–260.
- [4] D. Kochkov, J. A. Smith, A. Alieva, Q. Wang, M. P. Brenner, S. Hoyer, Machine learning–accelerated computational fluid dynamics, *Proceedings of the National Academy of Sciences* 118 (21) (2021) e2101784118.
- [5] Y. Bar-Sinai, S. Hoyer, J. Hickey, M. P. Brenner, Learning data-driven discretizations for partial differential equations, *Proceedings of the National Academy of Sciences* 116 (31) (2019) 15344–15349.
- [6] B. Stevens, T. Colonius, Enhancement of shock-capturing methods via machine learning, *Theoretical and Computational Fluid Dynamics* 34 (4) (2020) 483–496.
- [7] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, Neural operator: Learning maps between function spaces with applications to pdes, *Journal of Machine Learning Research* 24 (89) (2023) 1–97.
- [8] I. E. Lagaris, A. Likas, D. I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, *IEEE Transactions on Neural Networks* 9 (5) (1998) 987–1000. doi:10.1109/72.712178. URL <https://doi.org/10.1109/72.712178>
- [9] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *Journal of Computational Physics* 378 (2019) 686–707. doi:10.1016/j.jcp.2018.10.045. URL <http://www.sciencedirect.com/science/article/pii/S0021999118307125>
- [10] L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis, Learning nonlinear operators via deepnet based on the universal approximation theorem of operators, *Nature machine intelligence* 3 (3) (2021) 218–229.

- [11] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Fourier neural operator for parametric partial differential equations, arXiv preprint arXiv:2010.08895 (2020).
- [12] A. Matveev, S. Ghosh, A. Hussain, J.-M. Leahy, M. Michaelides, Lightweight diffusion multiplier and uncertainty quantification for fourier neural operators (2025). arXiv:2508.00643.
URL <https://arxiv.org/abs/2508.00643>
- [13] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, Physics-informed machine learning, *Nature Reviews Physics* 3 (6) (2021) 422–440.
- [14] R. Zhang, Y. Liu, H. Sun, Physics-informed multi-lstm networks for metamodeling of nonlinear structures, *Computer Methods in Applied Mechanics and Engineering* 369 (2020) 113226.
- [15] Z. Uddin, S. Ganga, R. Asthana, W. Ibrahim, Wavelets based physics informed neural networks to solve non-linear differential equations, *Scientific Reports* 13 (1) (2023) 2882.
- [16] B. Moseley, A. Markham, T. Nissen-Meyer, Solving the wave equation with physics-informed deep learning, arXiv preprint arXiv:2006.11894 (2020).
- [17] B. Moseley, A. Markham, T. Nissen-Meyer, Finite basis physics-informed neural networks (fbpinns): a scalable domain decomposition approach for solving differential equations, *Advances in Computational Mathematics* 49 (4) (2023) 62.
- [18] A. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, M. W. Mahoney, Characterizing possible failure modes in physics-informed neural networks, *Advances in neural information processing systems* 34 (2021) 26548–26560.
- [19] J. C. Wong, C. C. Ooi, A. Gupta, P.-H. Chiu, J. S. Z. Low, M. H. Dao, Y.-S. Ong, The baldwin effect in advancing generalizability of physics-informed neural networks, arXiv e-prints (2023) arXiv–2312.

- [20] Y. Wang, J. Bai, M. S. Eshaghi, C. Anitescu, X. Zhuang, T. Rabczuk, Y. Liu, Transfer learning in physics-informed neural networks: Full fine-tuning, lightweight fine-tuning, and low-rank adaptation, *International Journal of Mechanical System Dynamics* (2025).
- [21] S. Goswami, C. Anitescu, S. Chakraborty, T. Rabczuk, Transfer learning enhanced physics informed neural network for phase-field modeling of fracture, *Theoretical and Applied Fracture Mechanics* 106 (2020) 102447.
- [22] L. Yuan, Y.-Q. Ni, X.-Y. Deng, S. Hao, A-pinn: Auxiliary physics informed neural networks for forward and inverse problems of nonlinear integro-differential equations, *Journal of Computational Physics* 462 (2022) 111260.
- [23] S. Desai, M. Mattheakis, H. Joy, P. Protopapas, S. Roberts, One-shot transfer learning of physics-informed neural networks, *Preprint* (2024).
- [24] Z. Zou, G. E. Karniadakis, L-hydra: Multi-head physics-informed neural networks (2023). arXiv:2301.02152.
URL <https://arxiv.org/abs/2301.02152>
- [25] W. Lei, P. Protopapas, J. Parikh, One-shot transfer learning for nonlinear odes (2023). arXiv:2311.14931.
URL <https://arxiv.org/abs/2311.14931>
- [26] S. Auroy, P. Protopapas, One-shot transfer learning for nonlinear pdes with perturbative pinns (2025). arXiv:2511.11137.
URL <https://arxiv.org/abs/2511.11137>
- [27] C. M. Bender, S. A. Orszag, *Advanced Mathematical Methods for Scientists and Engineers I: Asymptotic Methods and Perturbation Theory*, Springer, New York, NY, 1999.
- [28] P. e. a. Virtanen, Scipy 1.0: Fundamental algorithms for scientific computing in python, *Nature Methods* 17 (3) (2020) 261–272. doi:10.1038/s41592-019-0686-2.
- [29] A. H. Nayfeh, *Perturbation Methods*, Wiley, New York, NY, 1973.

- [30] J. N. Kutz, Advanced differential equations: Asymptotics & perturbations, arXiv preprint arXiv:2012.14591 (2020).
- [31] S. Ghotekar, R. K. Sharma, Halo orbits in the photo-gravitational restricted three-body problem, *International Journal of Astronomy and Astrophysics* 9 (3) (2019) 274–291.
- [32] D. L. Richardson, Analytic construction of periodic orbits about the collinear points, *Celestial mechanics* 22 (3) (1980) 241–253.
- [33] M. Avey, F. Tornabene, N. M. Aslanova, A. H. Sofiyev, The application of the modified lindstedt–poincaré method to solve the nonlinear vibration problem of exponentially graded laminated plates on elastic foundations, *Mathematics* 12 (5) (2024) 749.
- [34] P. Amore, F. M. Fernández, On the application of the lindstedt–poincaré method to the lotka–volterra system, *Annals of Physics* 396 (2018) 293–303.
- [35] V. Dwivedi, B. Srinivasan, Physics informed extreme learning machine (pielm)—a rapid method for the numerical solution of partial differential equations, *Neurocomputing* 391 (2020) 96–118.
- [36] J. W. van Beek, V. Dolean, B. Moseley, Local feature filtering for scalable and well-conditioned Random Feature Methods, *Computer Methods in Applied Mechanics and Engineering* 449 (2025) 118583. doi:10.1016/J.CMA.2025.118583.
URL <https://www.sciencedirect.com/science/article/pii/S0045782525008552>
- [37] A. Arzani, K. W. Cassel, R. M. D’Souza, Theory-guided physics-informed neural networks for boundary layer problems with singular perturbation, *Journal of Computational Physics* 473 (2023) 111768.
- [38] S. Wang, P. Zhao, Q. Ma, T. Song, General-kindred physics-informed neural network to the solutions of singularly perturbed differential equations, *Physics of Fluids* 36 (11) (2024).
- [39] L. Wang, L. Zhang, G. He, Chien-physics-informed neural networks for solving singularly perturbed boundary-layer problems, *Applied Mathematics and Mechanics* 45 (9) (2024) 1467–1480.

- [40] N. Ohashi, N. P. Nguyen, L. K. Hwang, B. Kwon, Multiple scale method integrated physics-informed neural networks for reconstructing transient natural convection, *Physics of Fluids* 36 (12) (2024).
- [41] A. Giorgilli, et al., Small denominators and exponential stability: From poincaré to the present time, *Rendiconti del Seminario Matematico e Fisico di Milano* 68 (1) (1998) 19–57.
- [42] J. C. Butcher, A history of runge-kutta methods, *Applied numerical mathematics* 20 (3) (1996) 247–260.
- [43] E. Hairer, G. Wanner, O. Solving, Ii: Stiff and differential-algebraic problems, Berlin [etc.]: Springer (1991).
- [44] S. Wang, H. Wang, P. Perdikaris, On the eigenvector bias of fourier feature networks: From regression to solving multi-scale pdes with physics-informed neural networks, *Computer Methods in Applied Mechanics and Engineering* 384 (2021) 113938.
- [45] V. Sitzmann, J. Martel, A. Bergman, D. Lindell, G. Wetzstein, Implicit neural representations with periodic activation functions, *Advances in neural information processing systems* 33 (2020) 7462–7473.
- [46] T. Grozdanovski, J. Shepherd, Approximating the periodic solutions of the lotka–volterra system., *The Proceedings of ANZIAM* 49 (2007) C243–C257.
- [47] X. Zou, Delay induced traveling wave fronts in reaction diffusion equations of kpp-fisher type, *Journal of computational and Applied Mathematics* 146 (2) (2002) 309–321.
- [48] É. Brunet, Some aspects of the fisher-kpp equation and the branching brownian motion, Ph.D. thesis, UPMC (2016).
- [49] W. J. Pierson Jr, Perturbation analysis of the navier-stokes equations in lagrangian form with selected linear solutions, *Journal of Geophysical Research* 67 (8) (1962) 3151–3160.
- [50] P. Tarancón-Álvarez, P. Tejerina-Pérez, R. Jimenez, P. Protopapas, Efficient pinns: Multi-head unimodular regularization of the solutions space, *arXiv preprint arXiv:2501.12116* (2025).

- [51] H. Wang, Y. Pu, S. Song, G. Huang, Advancing generalization in pinns through latent-space representations, IEEE Transactions on Neural Networks and Learning Systems (2025).
- [52] P. Amore, A. Aranda, Improved lindstedt–poincaré method for the solution of nonlinear problems, Journal of Sound and Vibration 283 (3-5) (2005) 1115–1136.
- [53] H. Hu, Z. Xiong, Comparison of two lindstedt–poincaré-type perturbation methods, Journal of Sound and Vibration 278 (1-2) (2004) 437–444.
- [54] M. Lara, On perturbation solutions in the restricted three-body problem dynamics, Acta Astronautica 195 (2022) 596–604.

Appendix A. Lindstedt–Poincaré method

Appendix A.1. Undamped Oscillator Derivation

This subsection presents the derivation of the Lindstedt–Poincaré method for the undamped oscillator with monomial nonlinearity. A nonlinear forced undamped oscillator is generally described by equation 4 for $\zeta = 0$. In the Lindstedt–Poincaré method, we perform the variable change $\tau = \omega t$ for the nonlinear equation. Substituting the series expansions for x and ω , each term in the rescaled nonlinear equation can be expressed up to n th-order as follows:

$$\begin{aligned}
1) \quad & \omega^2 \ddot{x} = \omega_0^2 \ddot{x}_0 + \cdots + \left(\sum_{k=0}^n \ddot{x}_{n-k} \sum_{i=0}^k \omega_i \omega_{k-i} \right) \varepsilon^n \\
2) \quad & \omega_0^2 x = \omega_0^2 x_0 + \cdots + \omega_0^2 x_n \varepsilon^n \\
3) \quad & \varepsilon x^q = \varepsilon x_0^q + \cdots + \varepsilon^n \sum_{\substack{k_0+k_1+\cdots+k_p=q \\ \sum_{i=0}^p i k_i=n-1}} \frac{q!}{k_0! k_1! \cdots k_p!} \prod_{i=0}^p x_i^{k_i} \\
4) \quad & \sum_{k=0}^m \Gamma_k \cos \left(\Omega_k \frac{\tau}{\omega} \right) \approx \sum_{k=0}^m \Gamma_k \cos \left(\Omega_k \frac{\tau}{\omega_0} \right)
\end{aligned}$$

From the expressions above, we observe that (1) can be decomposed into three terms: one depending on ω_n , another on $\ddot{x}_n$ and a final term involving only quantities of order lower than n :

$$1) \quad \omega^2 \ddot{x} = \omega_0^2 \ddot{x}_0 + \cdots + \left[2\omega_0 \omega_n \ddot{x}_0 + \ddot{x}_n \omega_0^2 + A(\ddot{x}_0, \ddot{x}_{n-1}, \omega_0, \dots, \omega_{n-1}) \right] \varepsilon^n$$

Combining all the expressions above, we obtain the general form of the n th-order differential equation for $n > 0$ can be described by equations A.1 and A.2. The left side of equation A.1 can be separated into a term that depends on ω_n and another that depends on previously computed quantities from lower orders.

$$\frac{d^2 u}{dt^2} + u_n = \mathcal{F}(u_0, \dots, u_{n-1}, \omega_0, \dots, \omega_{n-1}) - 2 \frac{\omega_n}{\omega_0} \frac{d^2 u_0}{dt^2} \quad (\text{A.1})$$

$$\begin{aligned} \mathcal{F}(x_0, \dots, x_{n-1}, \omega_0, \dots, \omega_{n-1}) = & \sum_{\substack{k_0 + \dots + k_p = q \\ \sum_{i=0}^p i k_i = n-1}} - \frac{q!}{\omega_0^2 k_0! k_1! \dots k_p!} \prod_{i=0}^p x_i^{k_i} \\ & - \frac{1}{\omega_0^2} A(\ddot{x}_0, \dots, \ddot{x}_{n-1}, \omega_0, \dots, \omega_{n-1}) \quad (\text{A.2}) \end{aligned}$$

To enforce the solvability condition, we calculate the integral of the forcing term in equation A.1 multiplied by the homogeneous solution of the system. Here, it is sufficient to project the forcing only onto $\cos \tau$, as the Lindstedt–Poincaré method guarantees the elimination of $\sin \tau$ components by imposing the condition $\dot{x} = 0$.

$$\int_0^{2\pi} \left(\mathcal{F}(x_0, \dots, x_{n-1}, \omega_0, \dots, \omega_{n-1}) - 2 \frac{\omega_n}{\omega_0} \ddot{x}_0 \right) \cdot \cos \tau = 0 \quad (\text{A.3})$$

After isolating ω_n , we obtain the final expression for calculating the frequency correction at each order:

$$\omega_n = \frac{\int_0^{2\pi} \mathcal{F}(u_0, \dots, u_{n-1}, \omega_0, \dots, \omega_{n-1}) \cdot \cos t \, dt}{\int_0^{2\pi} -\frac{2\ddot{x}_0}{\omega_0} \cos t \, dt} \quad (\text{A.4})$$

Appendix A.2. Equilibrium-centered Lotka–Volterra Derivation

In this subsection, we follow Grozdanovski’s weakly nonlinear formulation of the Lotka–Volterra system [46]. Section 4.3 introduced this formulation,

where η and ξ are defined as corrections to the non-trivial critical point $(1, 1)$, following $x(t) = 1 + \varepsilon \xi(t)$ and $y(t) = 1 + \varepsilon \eta(t)$. Inserting these definitions into 14, we obtain the following system:

$$\begin{cases} \frac{d\xi}{dt} = -\eta(t)(1 + \varepsilon \xi(t)), \\ \frac{d\eta}{dt} = \alpha \xi(t)(1 + \varepsilon \eta(t)), \end{cases} \quad (\text{A.5})$$

Using the Lindstedt–Poincaré method, we introduce the new time variable $\tau = \omega t$, which rescales time and leads to the new system:

$$\begin{cases} \omega \frac{d\xi}{d\tau} = -\eta(\tau)(1 + \varepsilon \xi(\tau)), \\ \omega \frac{d\eta}{d\tau} = \alpha \xi(\tau)(1 + \varepsilon \eta(\tau)), \end{cases} \quad (\text{A.6})$$

Following the usual perturbation approach, we expand all quantities as power series in the small parameter ε :

$$\begin{aligned} \xi(\tau) &= \xi_0 + \varepsilon \xi_1 + \varepsilon^2 \xi_2 + \cdots \\ \eta(\tau) &= \eta_0 + \varepsilon \eta_1 + \varepsilon^2 \eta_2 + \cdots \\ \omega &= \sqrt{\alpha} + \varepsilon \omega_1 + \varepsilon^2 \omega_2 + \cdots \end{aligned}$$

By inserting these expressions into A.6, we obtain that the general formula for the n th-order correction is:

$$\begin{cases} \omega_0 \frac{d\xi_n}{d\tau} + \eta_n = -\omega_n \frac{d\xi_0}{d\tau} - \sum_{i=1}^{n-1} \omega_i \frac{d\xi_{n-i}}{d\tau} - \sum_{i=0}^{n-1} \xi_i \eta_{n-1-i} \\ \omega_0 \frac{d\eta_n}{d\tau} - \alpha \xi_n = -\omega_n \frac{d\eta_0}{d\tau} - \sum_{i=1}^{n-1} \omega_i \frac{d\eta_{n-i}}{d\tau} + \alpha \sum_{i=0}^{n-1} \xi_i \eta_{n-1-i} \end{cases} \quad (\text{A.7})$$

To avoid resonance, the solvability conditions must be satisfied, which implies

$$\int_0^{2\pi} F_{\xi_n}(t) \eta_0(t) dt = 0, \quad \int_0^{2\pi} F_{\eta_n}(t) \xi_0(t) dt = 0$$

where F_{ξ_n} and F_{η_n} denote the forcing terms in the equations for ξ_n and η_n , respectively. When one of these conditions is satisfied, so is the other. We choose to project η_0 and after algebraic manipulation of the forcing function, which is represented in A.7, we find that ω_n is given by:

$$\omega_n = \frac{\int \left(\sum_{i=1}^{n-1} \omega_i \frac{d\xi_{n-i}}{d\tau} + \sum_{i=0}^{n-1} \xi_i \eta_{n-1-i} \right) \cdot \eta_0 d\tau}{\int_0^{2\pi} -\frac{d\xi_0}{d\tau} \cdot \eta_0 d\tau} \quad (\text{A.8})$$

Since we have an expression for ω_n , we can follow the same approach that was outlined for the undamped oscillator to solve the Lindstedt–Poincaré system that approximates the equilibrium-centered Lotka–Volterra system.

Appendix B. Pretraining configurations

In this section, we describe the pretraining configurations of the PTL-PINNs models used to obtain the previous results.

Appendix B.1. ODEs

For each damping regime, we present the general expected forcing function in the perturbative system and use it to guide the choice of forcing functions in pretraining. Table B.3 shows the configurations of each train head for the undamped model and the equilibrium-centered Lotka–Volterra system. For these equations, the general functional form of the terms present in the forcing of the perturbative system is given by equation B.1. By expressing the undamped oscillator as a system of two first-order ODEs, it takes the same matrix form as the Lotka–Volterra system. Since both systems admit solutions of the same functional form, we use the same model to solve both equations.

$$\sum_{k=0}^m \Gamma_k \cos(\Omega_k t), \quad \Gamma_k = \{a_i\}_{i=1}^m \quad a_i \sim \mathcal{U}\left(0, \frac{2}{m}\right) \quad (\text{B.1})$$

Table B.3: Pretraining configurations of the undamped and Lotka–Volterra PTL-PINN.

Head	Ω_k	Γ_k	$(x(0), \dot{x}(0))$	ω_0
0	$\emptyset$	$\emptyset$	(1, 0)	1
1	$\emptyset$	$\emptyset$	(1.5, 0)	1
2	$\emptyset$	$\emptyset$	(0.5, 0)	1
3	{3}	{1}	(1.2, 0)	1
4	{3}	{1}	(0.5, 0)	1
5	{6}	{1}	(1, 0)	1
6	{3, 12}	$\mathcal{U}(0, 1)$	(0, 0)	1
7	{2, 4, 5}	$\mathcal{U}(0, 2/3)$	(0, 0)	1
8	{2, 3, 4, 5, 6}	$\mathcal{U}(0, 2/5)$	(0, 0)	1
9	{3, 6, 9, 12, 15}	$\mathcal{U}(0, 2/5)$	(0, 0)	1
10	{6, 9, 18, 21}	$\mathcal{U}(0, 2/4)$	(0, 0)	1
11	{3, 6, 9, 12, 15, 18, 21, 24}	$\mathcal{U}(0, 2/8)$	(0, 0)	1

Table B.4 shows the configurations of each train head for the under-damped model. In this damping regime, the general functional form of the terms present in the forcing of the perturbative system is given by:

$$\sum_{k=0}^m \Gamma_k e^{-\mu \zeta \omega_0 t} \cos(\Omega_k \sqrt{1 - \zeta^2} t + \phi), \quad \Gamma_k = \{a_i\}_{i=1}^m \quad a_i \sim \mathcal{U}\left(0, \frac{2}{m}\right) \quad (\text{B.2})$$

Table B.4: Pretraining configurations of the underdamped PTL-PINN model.

Head	ζ	Ω_k	Γ_k	$(x(0), \dot{x}(0))$	ω_0	μ
0	0.05	$\emptyset$	$\emptyset$	(1, 0)	1	0
1	0.10	$\emptyset$	$\emptyset$	(1, 0)	1	0
2	0.20	$\emptyset$	$\emptyset$	(1, 0)	1	0
3	0.05	{3}	{1}	(1.2, 0)	1	0
4	0.10	{3}	$\{-\frac{1}{4}\}$	(0.5, 0)	1	0
5	0.10	{6}	{1}	(1, 0)	1	0
6	0.50	{3, 12}	$\{-\frac{1}{2}, -\frac{1}{2}\}$	(0, 0)	1	0
7	0.05	{2, 4, 5}	$\mathcal{U}(0, 2/3)$	(0, 0)	1	0
8	0.20	{2, 3, 4, 5, 6}	$\mathcal{U}(0, 2/5)$	(0, 0)	1	0
9	0.40	{3, 6, 9, 12, 15}	$\mathcal{U}(0, 2/5)$	(0, 0)	1	0
10	0.10	{6, 9, 18, 21}	$\mathcal{U}(0, 2/4)$	(0, 0)	1	0
11	0.05	{3, 6, 9, 12, 15, 18, 21, 24}	$\mathcal{U}(0, 2/8)$	(0, 0)	1	0

Table B.5 shows the configurations of each train head for the overdamped model. In this damping regime, the general functional form of the terms present in the forcing of the perturbative system is given by:

$$\Gamma(e^{-\mu_1\lambda_1 t} + e^{-\mu_2\lambda_1\lambda_2 t} + e^{-\mu_3\lambda_1\lambda_2 t} + e^{-\mu_4\lambda_1\lambda_2 t}) \cos(\Omega t + \phi), \quad \Gamma \sim \mathcal{U}(0, 2) \quad (\text{B.3})$$

Table B.5: Pretraining configurations of the overdamped PTL-PINN model.

Head	ζ	$\boldsymbol{\mu} = (\mu_1, \mu_2, \mu_3, \mu_4)$	$(x(0), \dot{x}(0))$	ω_0	Ω_k	Γ_k
1	5	(0, 0, 0, 0)	(2.0, 0)	1	{1}	$\mathcal{U}(0, 2)$
2	10	(0, 0, 0, 0)	(1.0, 0)	1	{1}	$\mathcal{U}(0, 2)$
3	20	(0, 0, 0, 0)	(1.0, 0)	1	{1}	$\mathcal{U}(0, 2)$
4	30	(0, 0, 0, 0)	(2.0, 0)	1	{1}	$\mathcal{U}(0, 2)$
5	40	(0, 0, 0, 0)	(2.0, 0)	1	{1}	$\mathcal{U}(0, 2)$
6	50	(0, 0, 0, 0)	(1.0, 0)	1	{1}	$\mathcal{U}(0, 2)$
7	60	(0, 0, 0, 0)	(1.0, 0)	1	{1}	$\mathcal{U}(0, 2)$
8	5	(1, 0, 0, 1)	(0.0, 0)	1	{1}	$\mathcal{U}(0, 2)$
9	30	(3, 0, 0, 3)	(0.0, 0)	1	{1}	$\mathcal{U}(0, 2)$
10	60	(3, 0, 0, 3)	(0.0, 0)	1	{1}	$\mathcal{U}(0, 2)$
11	10	(3, 2, 1, 3)	(1.5, 0)	1	{1}	$\mathcal{U}(0, 2)$
12	30	(3, 1, 2, 3)	(1.0, 0)	1	{1}	$\mathcal{U}(0, 2)$
13	20	(3, 1, 2, 3)	(1.0, 0)	1	{1}	$\mathcal{U}(0, 2)$
14	40	(3, 0, 0, 3)	(2.0, 0)	1	{3}	$\mathcal{U}(0, 2)$
15	20	(3, 0, 0, 3)	(1.0, 0)	1	{5}	$\mathcal{U}(0, 2)$

Appendix B.2. PDEs

Each linear equation of the perturbative system for the KPP–Fisher is a forced 1D Heat equation. The zeroth-order solution for the initial condition $u(x, 0) = \sin(\pi x/L)$ can easily be obtained from separation of variables to be:

$$u_0 = e^{-D(\frac{\pi}{L})^2 t} \sin\left(\frac{\pi x}{L}\right) \quad (\text{B.4})$$

Table B.6 presents the pretraining configurations for the KPP–Fisher equation. More generally, we also consider the forcing terms of the following type:

$$e^{-\alpha \lambda_n t} \sin(n\pi x/L), \quad \lambda_n = D\left(\frac{n\pi}{L}\right)^2 \quad (\text{B.5})$$

Table B.6: Pretraining configurations of the KPP–Fisher PTL-PINN model.

Head	D	$u(x, 0)$	$f(x, t)$
0	0.1	$\sin\left(\frac{\pi x}{L}\right)$	0
1	0.05	$\sin\left(\frac{\pi x}{L}\right)$	0
2	0.01	$\sin\left(\frac{\pi x}{L}\right)$	0
3	0.1	0	$u_0(1 - u_0)$
4	0.05	0	$u_0(1 - u_0)$
5	0.01	0	$u_0(1 - u_0)$
6	0.50	0	$n = 1, \alpha = 1$
7	0.1	0	$n = 2, \alpha = 2$
8	0.05	0	$n = 1, \alpha = 2$

Similarly, each equation in the perturbative system for the Wave equation corresponds to a linear Wave equation. Table B.7 shows the configurations used for the pretraining of this PTL-PINN model. The zeroth-order solution for initial condition $u(x, 0) = \sin(\pi x/L)$ is:

$$u_0(x, t) = \sin\left(\frac{\pi x}{L}\right) \cos\left(\frac{c\pi t}{L}\right) \quad (\text{B.6})$$

More generally, we can consider forcing functions with arbitrary wave number k and frequency ω :

$$\sin\left(\frac{k\pi x}{L}\right) \cos\left(\frac{c\omega\pi t}{L}\right), \quad \omega, k \in \mathbb{N} \quad (\text{B.7})$$

Table B.7: Pretraining configurations of the Wave PTL-PINN model.

Head	c	$u(x, 0)$	$f(x, t)$
0	1	$\sin\left(\frac{\pi x}{L}\right)$	0
1	0.8	$\sin\left(\frac{\pi x}{L}\right)$	0
2	1.2	$\sin\left(\frac{\pi x}{L}\right)$	0
3	1	0	$u_0(1 - u_0)$
4	0.8	0	$u_0(1 - u_0)$
5	1.2	0	$u_0(1 - u_0)$
6	1	0	$\omega = 1, k = 1$
7	0.8	0	$\omega = 2, k = 2$
8	1.2	0	$\omega = 1, k = 2$